

Maschinelles Lernen

6. Training von neuronalen Netzen

Prof. Dr. Dennis Janka

Hochschule Karlsruhe
University of
Applied Sciences



**Watching
a model train**



**Watching
a model train**



Ausgangssituation: Training von neuronalen Netzen

Erinnerung: Wir suchen diejenigen Matrizen $\mathbf{W}^{(i)}$ und Vektoren $\mathbf{b}^{(i)}$ für die Schichten des Netzes, so dass die Verlustfunktion auf dem Trainingsset minimal wird. Im Folgenden stellen wir uns zunächst vor, dass alle Einträge von $\mathbf{W}^{(i)}$ und $\mathbf{b}^{(i)}$ in einem (potentiell sehr großen) Vektor $\mathbf{w}$ angeordnet sind.

Aufgabe: Finde denjenigen Vektor $\mathbf{w}$, so dass die Verlustfunktion minimal wird.
Allgemein:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \underbrace{l(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))}_{\text{Verlust}}$$

Verlust (=Abweichung zwischen Wahrheit und Vorhersage) für ein Trainingsbeispiel (z.B. MSE, Kreuzentropie oder andere Verlustfunktionen)

Überblick

6.1 Gradientenabstieg

6.2 Stochastischer Gradientenabstieg

6.3 Berechnung der Gradienten (Backpropagation)

6.4 Probleme (und Lösungsansätze) beim Trainieren:

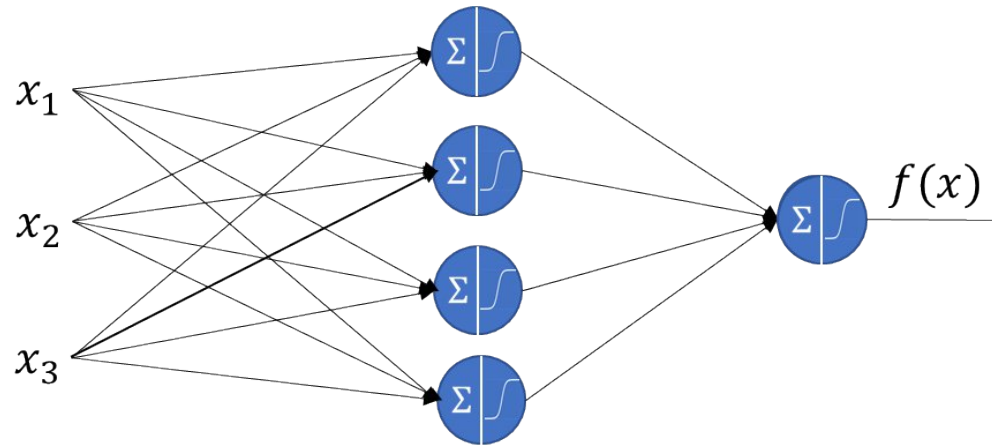
- Vanishing gradient Problem
- Wahl der Lernrate
- Regularisierung für neuronale Netze

6.1 Gradientenabstieg

Erinnerung: Supervised Learning als Suche nach Funktionen

Jedes **neuronale Netz** ist eine Funktion:

Trainingsbeispiel $\mathbf{x} \rightarrow$ Vorhersage $f(\mathbf{x})$



$$f(\mathbf{x}) = \sigma(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x}))$$

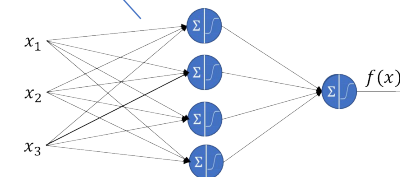
Erinnerung: Supervised Learning als Suche nach Funktionen

Jede **Verlustfunktion** ist eine Funktion:

Koeffizientenvektor $\mathbf{w}$
(Trainingsdaten $(\mathbf{x}_i, \mathbf{y}_i)$) $\rightarrow$ Verlust auf Trainingsdatensatz $L(\mathbf{w})$

Trainingsdaten sind
fest, sind vor allem
an Abhängigkeit von
 w interessiert

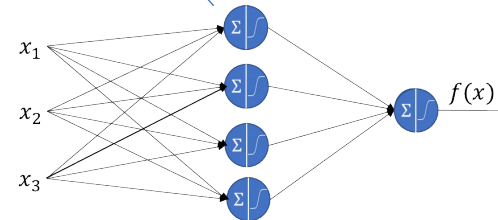
$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}_i, \underbrace{f(\mathbf{x}_i; \mathbf{w})}_{\text{Neural Network}})$$



Optimierung der Verlustfunktion

Beim Trainieren von neuronalen Netzen soll die Verlustfunktion auf den Trainingsdaten minimiert werden:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, \underbrace{f(\mathbf{x}_i; \mathbf{w})}_{\text{NN}})$$



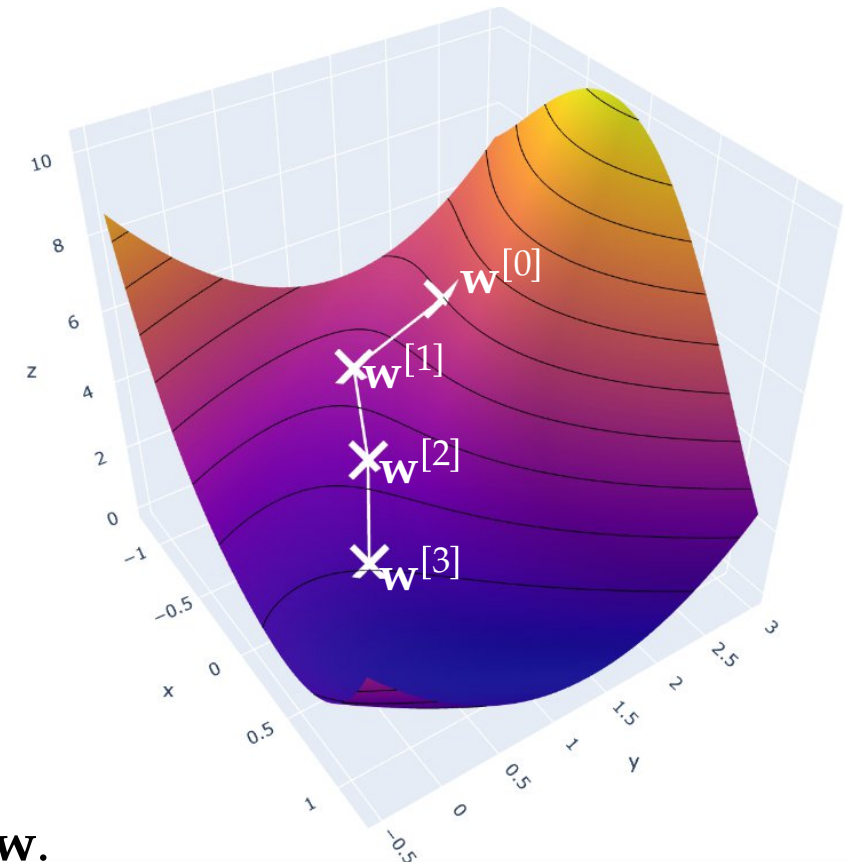
Gradientenabstieg für die Verlustfunktion

Man startet mit irgendeiner Schätzung für die Koeffizienten und verbessert diese sukzessive.

1. Starte mit einer Schätzung der Lösung $\mathbf{w}^{[0]}$
2. Für $k = 0, 1, 2, \dots$:
 - a) $\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} - \alpha^{[k]} \nabla L(\mathbf{w}^{[k]})^T, \quad \alpha^{[k]} > 0$
 - b) Stop, falls Abbruchbedingung erfüllt
(maximale Anzahl Iterationen, wenig Fortschritt, ...)

Mit Lernrate $\alpha^{[k]} \in \mathbb{R}$.

Beachte: Jeder „Ort“ (Längen-/Breitengrad) in dem Gebirge ist ein anderer Vektor $\mathbf{w}$ von Gewichten für das neuronale Netz. Die Höhe ist der Gesamtverlust (=Funktionswert) für das gegebene $\mathbf{w}$.

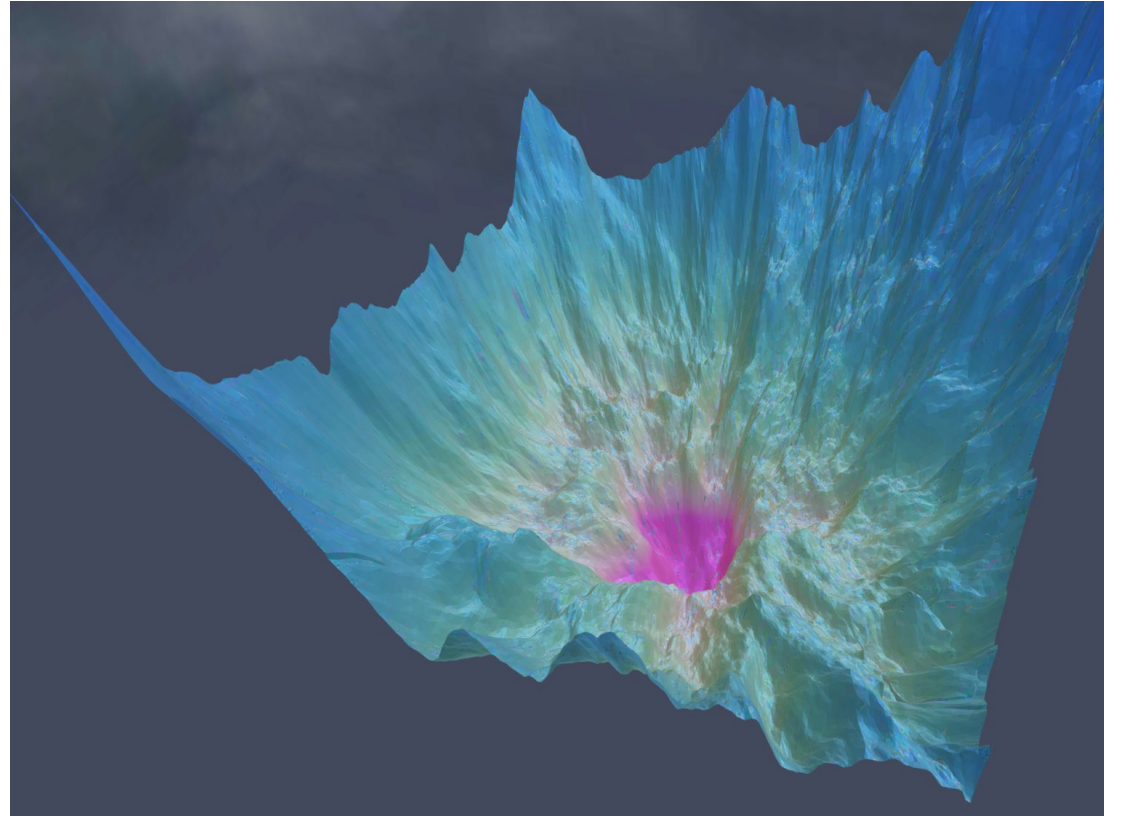


Einschub: Loss Landscapes

In Wirklichkeit hat man natürlich keine „Gebirge“, sondern sehr hochdimensionale Räume, die sehr schwer vorstellbar sind.

Auf <https://losslandscape.com/> gibt es interessante Visualisierungen von Verlustfunktionen.

Idee: Projektion der hochdimensionalen Verlustfunktion auf *zufällig* ausgewählte Ebene, dann Sampling der Funktionswerte auf einem Gitter in der Umgebung des Minimums.



Gradientenabstieg: Beispiel (zunächst ohne NN, sondern lineares Modell)

- Beispiel: Lineare Regression mit zwei Features, $N = 100$ Trainingsbeispiele
- Wir minimieren die Funktion $L: \mathbb{R}^2 \rightarrow \mathbb{R}$:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{100} \sum_{i=1}^{100} (y_i - (w_1 x_{i,1} + w_2 x_{i,2}))^2$$

x_1	x_2	y
0.9150	0.4414	3.5857
0.9593	0.3423	3.4530
0.6009	0.9040	3.7469
0.7936	0.8090	3.1960
0.1332	0.8317	2.4604
0.5936	0.2695	2.9890
0.5677	0.1994	2.1253
...	...	...

100 Datenpunkte

Beispiel: Trainingsdaten
(2 Features, 1D-Label)

Beispiel Gradientenabstieg: Schritte

Starte mit einer Schätzung $\mathbf{w}^{[0]} = (w_1^{[0]}, w_2^{[0]})^T$. Wähle $\alpha^{[0]}$. Setze Epoche $k = 1$.

1. Bestimme Gradient (Suchrichtung) bei $w^{[0]}$:

$$\begin{aligned}\mathbf{d}^{[0]} = -\nabla L(\mathbf{w}^{[0]}) &= -\nabla \left(\frac{1}{100} \sum_{i=1}^{100} \underbrace{\left(y_i - (w_1^{[0]} x_{i,1} + w_2^{[0]} x_{i,2}) \right)^2}_{r_i: \text{Residuum für } i\text{-ten Trainingsdatenpunkt, hängt von } w \text{ ab!}} \right) = -\nabla \left(\frac{1}{100} \sum_{i=1}^{100} r_i^2 \right) \\ &= -\frac{1}{100} \sum_{i=1}^{100} \nabla r_i = -\frac{1}{100} \sum_{i=1}^{100} 2r_i \left(\frac{\partial r_i}{\partial \mathbf{w}} \right)^T = \frac{1}{100} \sum_{i=1}^{100} 2r_i \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in \mathbb{R}^2\end{aligned}$$

2. Neue Iterierte:

$$\mathbf{w}^{[1]} = \mathbf{w}^{[0]} + \alpha^{[0]} \mathbf{d}^{[0]}$$

3. Wähle neue Schrittweite $\alpha^{[1]}$, setze Epoche $k = 2$.

Beispiel Gradientenabstieg: Schritte

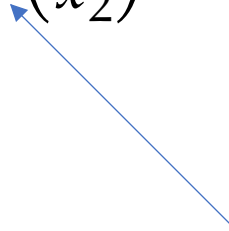
3. Bestimme Suchrichtung bei $\mathbf{w}^{[1]}$:

$$\mathbf{d}^{[1]} = -\nabla L(\mathbf{w}^{[1]}) = \frac{1}{100} \sum_{i=1}^{100} 2r_i \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

4. Neue Iterierte: $\mathbf{w}^{[2]} = \mathbf{w}^{[1]} + \alpha^{[1]} \mathbf{d}^{[1]}$

5. Wähle neue Schrittweite $\alpha^{[2]}$, setze Epoche $k = 3$.

r_i hängt von $\mathbf{w}^{[1]}$ ab



Usw.

6.2 Stochastischer Gradientenabstieg

Minimierung des Generalisierungsfehlers

Erinnerung: Minimierung der Verlustfunktion

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$$

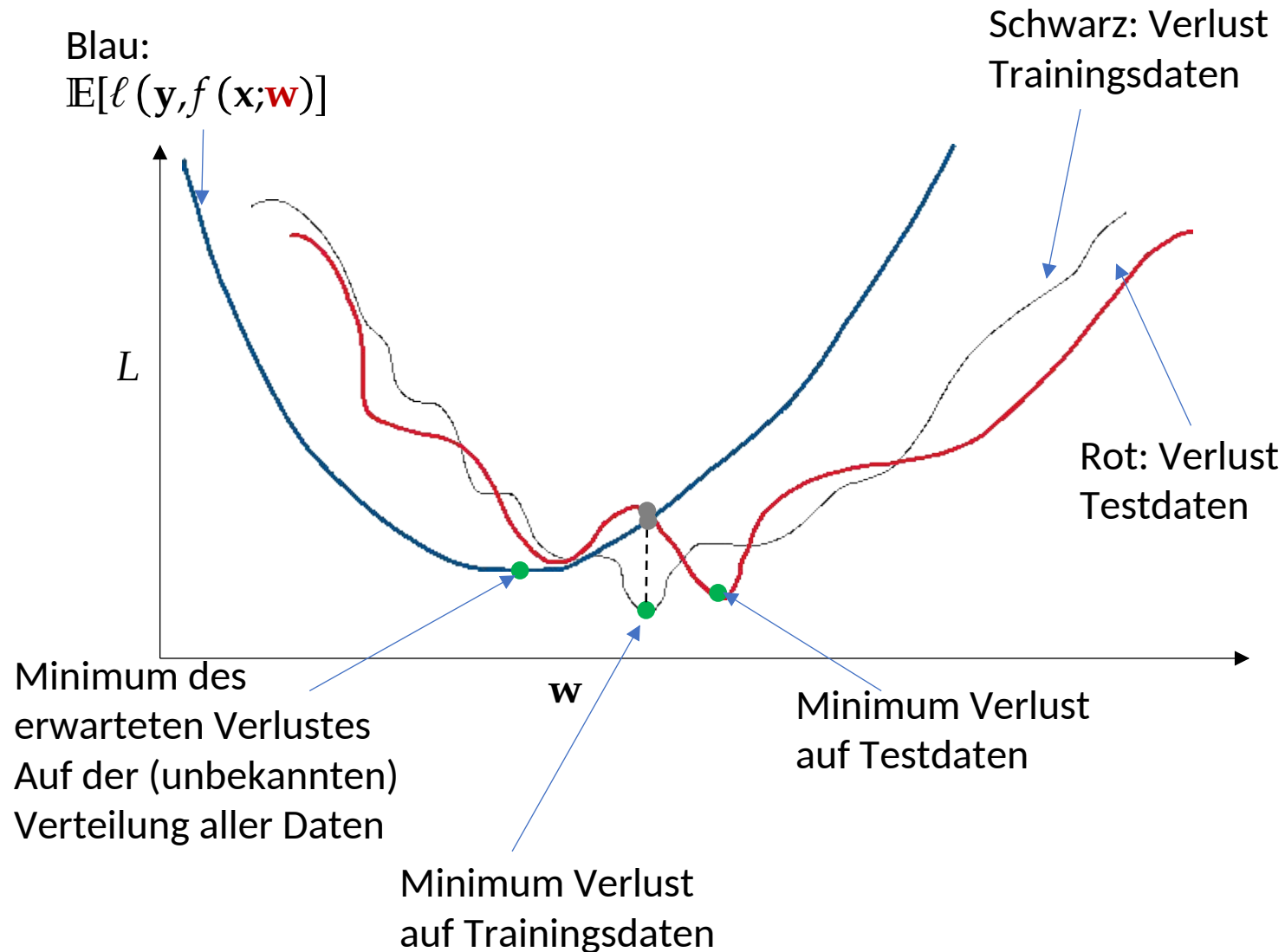
ist nur eine Approximation für das eigentliche Ziel.

Eigentliches Ziel/Minimierungsproblem:

$$\min_{\mathbf{w}} \mathbb{E}[\ell(\mathbf{y}, f(\mathbf{x}; \mathbf{w}))]$$

(minimiere erwarteten Verlust auf *allen möglichen* Feature-Label-Paaren
—> Generalisierungsfehler)

Illustration Training/Test/unbekannte Daten



- Für jeden Datensatz erhält man eine etwas andere Verlustfunktion $L(\mathbf{w})$
- Overfitting bedeutet, die gefundenen Parameter $\mathbf{w}$ minimieren nur eine bestimmte Instanz von $L(\mathbf{w})$ (die der Trainingsdaten), aber nicht den erwarteten Fehler auf allen möglichen (ungesehenen) Daten
- Regularisierung wird eingesetzt, um Overfitting zu verhindern (auch das ändert die Verlustfunktion!)
- Zusammenfassend: wir suchen nicht das Minimum der Verlustfunktion auf den Trainingsdaten, sondern hoffen, dass es eine gute Approximation für das Minimum von $\mathbb{E}[\ell(\mathbf{y}, f(\mathbf{x}; \mathbf{w}))]$ ist.

Minimierung von $\mathbb{E}$:

Stochastischer Gradientenabstieg (SGD)

Ziel: (lokales) Minimum von $\min_{\mathbf{w}} \mathbb{E}[\ell(\mathbf{y}, f(\mathbf{x}; \mathbf{w}))]$

Algorithmus:

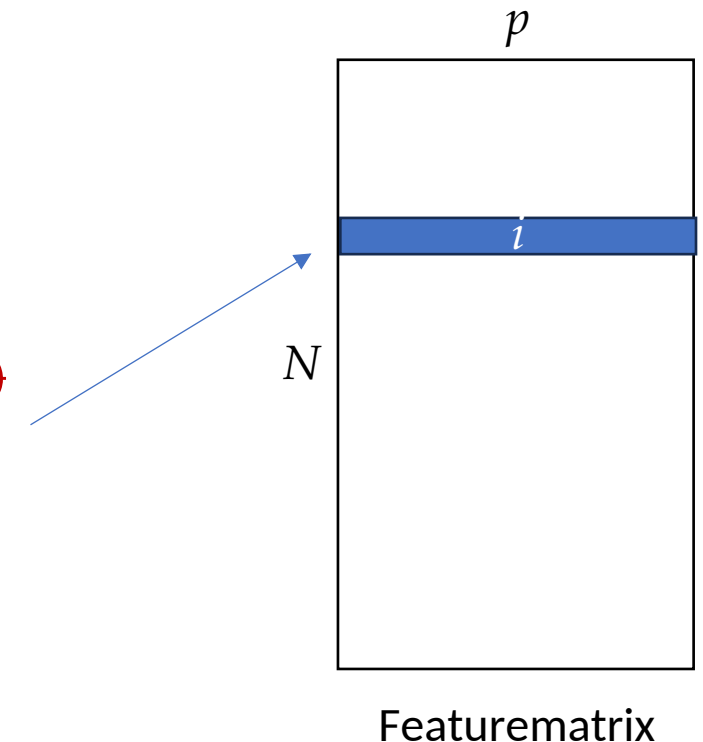
1. Starte mit einer Schätzung der Lösung $\mathbf{w}^{[0]}$
2. Für $k = 0, 1, 2, \dots$:
 - a) Generiere zufälligen Datenpunkt (Feature-Vektor-Paar) $(\mathbf{x}, \mathbf{y})$
 - b) $\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} - \alpha^{[k]} \nabla_{\mathbf{w}} \ell(\mathbf{y}, f(\mathbf{x}; \mathbf{w}))$, $\alpha^{[k]} > 0$
 - c) Stop, falls Abbruchbedingung erfüllt
(maximale Anzahl Iterationen, wenig Fortschritt, ...)

Minimierung von $\mathbb{E}$: Stochastischer Gradientenabstieg

Ziel: (lokales) Minimum von $\min_{\mathbf{w}} \mathbb{E}[\ell(\mathbf{y}, f(\mathbf{x}; \mathbf{w}))]$

Algorithmus:

1. Starte mit einer Schätzung der Lösung $\mathbf{w}^{[0]}$
2. Für $k = 0, 1, 2, \dots$:
 - a) ~~Generiere zufälligen Datenpunkt (Feature-Vektor-Paar) $(\mathbf{x}, \mathbf{y})$~~
 - b) Wähle zufälligen Datenpunkt $(\mathbf{x}_i, \mathbf{y}_i)$ aus Trainingsdaten
 - c) $\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} - \alpha^{[k]} \nabla_{\mathbf{w}} \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$, $\alpha^{[k]} > 0$
 - d) Stop, falls Abbruchbedingung erfüllt
(maximale Anzahl Iterationen, wenig Fortschritt, ...)



Vergleich der Gradienten (=Suchrichtungen)

Gradientenabstieg (auch: Batch Gradientenabstieg):

- $\nabla_{\mathbf{w}} L(\mathbf{w}) = \nabla_{\mathbf{w}} \left(\frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w})) \right)$: Ableitung der Summe
= $\frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$: Summe der Ableitungen

Stochastischer Gradientenabstieg:

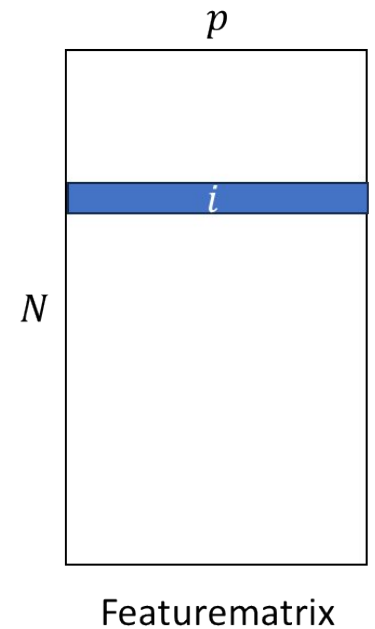
- $\nabla_{\mathbf{w}} \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$: Ableitung *eines* Summanden

Dimension ist in beiden Fällen gleich, nämlich p (=Anzahl der Koeffizienten $\mathbf{w}$)

Stochastisches Gradientenverfahren: Praktische Umsetzung

1. Randomisiere Trainingsbeispiele (zufällige Reihenfolge der Zeilen der Featurematrix)
2. Starte mit einer Schätzung der Lösung $\mathbf{w}^{[0]}$
3. Für $k = 0, 1, 2, \dots$:
 - a) Setze $\mathbf{w}^{[k+1,0]} = \mathbf{w}^{[k]}$, wähle Lernrate $\alpha^{[k]}$
 - b) Für $i = 1, \dots, N$
 - a) $\mathbf{w}^{[k+1,i]} = \mathbf{w}^{[k+1,i-1]} - \alpha^{[k]} \nabla_{\mathbf{w}} \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$, $\alpha^{[k]} > 0$
 - c) Setze $\mathbf{w}^{[k+1]} = \mathbf{w}^{[k+1,N]}$
 - d) Stop, falls $\mathbf{w}^{[k+1]}$ Abbruchbedingung erfüllt

- Innere Schleife (i) läuft über Zeilen der Featurematrix (Summanden von L)
- Äußere Schleife (k) entspricht den Iterationen des normalen Gradientenverfahrens („Epochen“)



Stochastischer Gradientenabstieg: Beispiel lineares Modell

- Beispiel: Lineare Regression mit zwei Features, $N = 100$ Trainingsbeispiele
- Stochastisches Gradientenverfahren für $L: \mathbb{R}^2 \rightarrow \mathbb{R}$:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{100} \sum_{i=1}^{100} (y_i - (w_1 x_{i,1} + w_2 x_{i,2}))^2$$

x_1	x_2	y
0.9150	0.4414	3.5857
0.9593	0.3423	3.4530
0.6009	0.9040	3.7469
0.7936	0.8090	3.1960
0.1332	0.8317	2.4604
0.5936	0.2695	2.9890
0.5677	0.1994	2.1253
...	...	...

100 Datenpunkte

Beispiel: Trainingsdaten
(2 Features, 1D-Label)

Beispiel Stochastischer Gradientenabstieg

Starte mit einer Schätzung $\mathbf{w}^{[0]} = (w_1^{[0]}, w_2^{[0]})^T$. Setze Epoche $k = 0$.

1. Erste Epoche: Setze $\mathbf{w}^{[1,0]} = \mathbf{w}^{[0]}$, wähle $\alpha^{[0]}$

2. Setze $i = 1$

Nächste Iterierte durch Gradienten des **1. Trainingsbeispiels**:

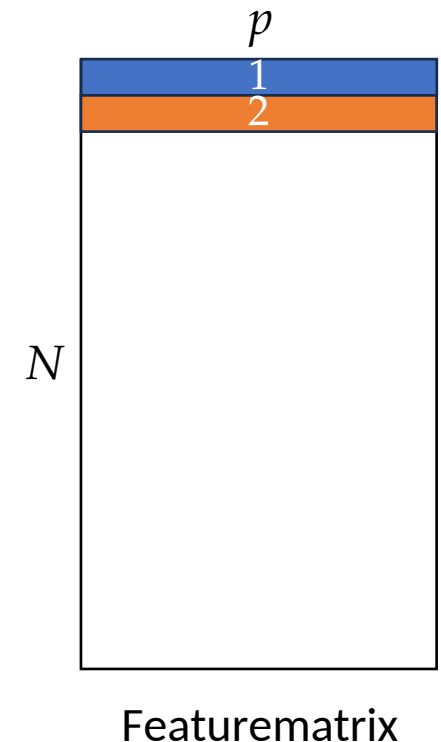
$$\mathbf{w}^{[1,1]} = \mathbf{w}^{[1,0]} - \alpha^{[0]} \nabla_{\mathbf{w}} \left((y_1 - (w_1 x_{1,1} + w_2 x_{1,2}))^2 \right)$$

3. Setze $i = 2$

Nächste Iterierte durch Gradienten des **2. Trainingsbeispiels**:

$$\mathbf{w}^{[1,2]} = \mathbf{w}^{[1,1]} - \alpha^{[0]} \nabla_{\mathbf{w}} \left((y_2 - (w_1 x_{2,1} + w_2 x_{2,2}))^2 \right)$$

4. ...



Beispiel Stochastischer Gradientenabstieg

...

101. Setze $i = 100$

Nächste Iterierte durch Gradienten des 100. Trainingsbeispiels:

$$\mathbf{w}^{[1,100]} = \mathbf{w}^{[1,99]} - \alpha^{[0]} \nabla_{\mathbf{w}} \left((y_2 - (w_1 x_{100,1} + w_2 x_{100,2}))^2 \right)$$

N

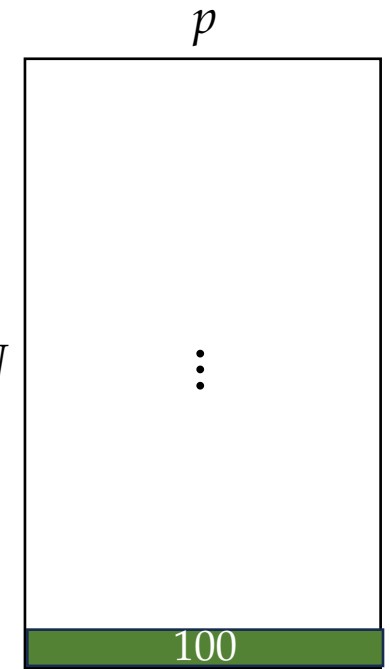
102. Setze $\mathbf{w}^{[1]} = \mathbf{w}^{[1,100]}$

103. Zweite Epoche: Wähle Lernrate $\alpha^{[1]}$, setze $\mathbf{w}^{[2,0]} = \mathbf{w}^{[1]}$

104. Setze $i = 1$

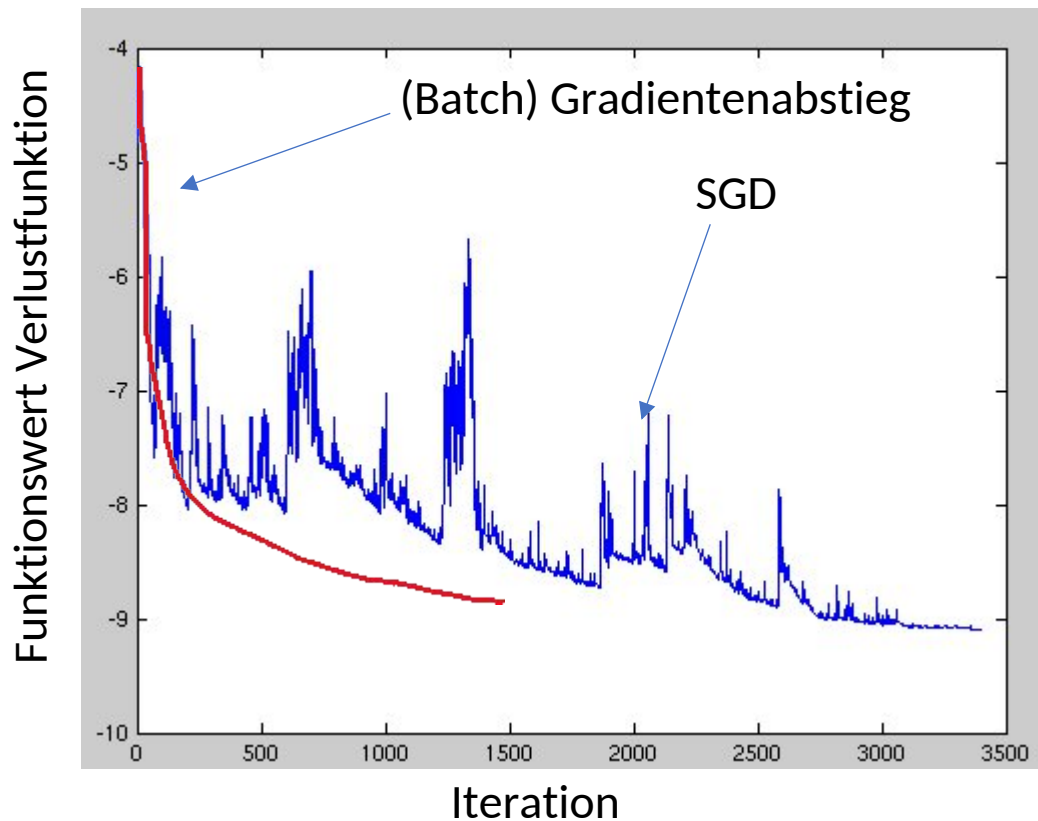
105. (Iteriere wieder über die 100 Zeilen der Featurematrix)

usw.



Featurematrix

Gradientenabstieg vs. stochastischer Gradientenabstieg



- Gradientenabstieg verbessert den Zielfunktionswert in jeder Iteration
- SGD verbessert den Zielfunktionswert auch, aber nur langfristig, d.h. im Erwartungswert (mit Rauschen)
- Gradientenabstieg benötigt weniger Schritte zum Konvergieren, aber jeder Schritt braucht viel mehr Rechenzeit
- Deshalb ist SGD in der Praxis schneller

Minibatch SGD

„Kompromiss“ zwischen (Batch) GD und SGD:

- Batch GD: Benutze alle Trainingsbeispiele zur Bestimmung der Suchrichtung
- SGD: Benutze ein Trainingsbeispiel zur Bestimmung der Suchrichtung
- Minibatch SGD: Benutze m Trainingsbeispiele zur Bestimmung der Suchrichtung ($1 < m < N$)

Minibatch ist **Schätzer** für Verlustfunktion:

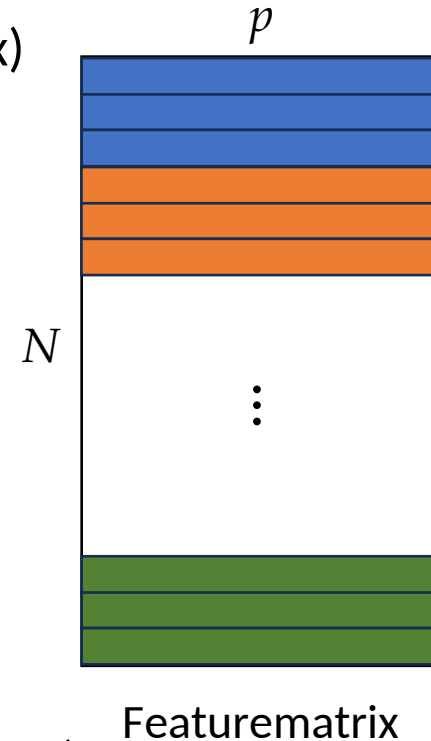
$$\frac{1}{m} \sum_{j=i_1}^{i_m} \ell(\mathbf{y}_j, f(\mathbf{x}_j; \mathbf{w})) \approx \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))$$

Gradient der Minibatch ist **Schätzer** für Gradient der Verlustfunktion (Suchrichtung!):

$$\nabla \left(\frac{1}{m} \sum_{j=i_1}^{i_m} \ell(\mathbf{y}_j, f(\mathbf{x}_j; \mathbf{w})) \right) \approx \nabla \left(\frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w})) \right)$$

Minibatch SGD

1. Randomisiere Trainingsbeispiele (zufällige Reihenfolge der Zeilen der Featurematrix)
2. Starte mit einer Schätzung der Lösung $\mathbf{w}^{[0]}$
3. Für $k = 0, 1, 2, \dots$:
 - a) Setze $\mathbf{w}^{[k+1,0]} = \mathbf{w}^{[k]}$, wähle Lernrate $\alpha^{[k]}$
 - b) Für $i = 1, \dots, N/m$
 - a) $\mathbf{w}^{[k+1,i]} = \mathbf{w}^{[k,i]} - \alpha^{[k]} \nabla_{\mathbf{w}} \sum_{j=(i-1)m}^{i \cdot m} \ell(\mathbf{y}_j, f(\mathbf{x}_j; \mathbf{w}))$, $\alpha^{[k]} > 0$
 - c) Setze $\mathbf{w}^{[k+1]} = \mathbf{w}^{[k+1,N]}$
 - d) Stop, falls $\mathbf{w}^{[k+1]}$ Abbruchbedingung erfüllt



- Innere Schleife (i) läuft über **Teilmenge der Zeilen** der Featurematrix (Summanden von L)
- Äußere Schleife (k) entspricht den Iterationen des normalen Gradientenverfahrens („Epochen“)

Minibatch SGD:

Nochmal das lineare Beispiel

- Beispiel: Lineare Regression mit zwei Features, $N = 100$ Trainingsbeispiele

- Stochastisches Gradientenverfahren für $L: \mathbb{R}^2 \rightarrow \mathbb{R}$:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{100} \sum_{i=1}^{100} (y_i - (w_1 x_{i,1} + w_2 x_{i,2}))^2$$

- Größe der Minibatch: $m = 10$

x_1	x_2	y
0.9150	0.4414	3.5857
0.9593	0.3423	3.4530
0.6009	0.9040	3.7469
0.7936	0.8090	3.1960
0.1332	0.8317	2.4604
0.5936	0.2695	2.9890
0.5677	0.1994	2.1253
...	...	...

100 Datenpunkte

Beispiel: Trainingsdaten
(2 Features, 1D-Label)

Beispiel Minibatch SGD

Starte mit einer Schätzung $\mathbf{w}^{[0]} = (w_1^{[0]}, w_2^{[0]})^T$. Setze Epoche $k = 0$.

1. Erste Epoche: Setze $\mathbf{w}^{[1,0]} = \mathbf{w}^{[0]}$, wähle $\alpha^{[0]}$

2. Setze $i = 1$

Nächste Iterierte durch Gradienten der **1. Minibatch**:

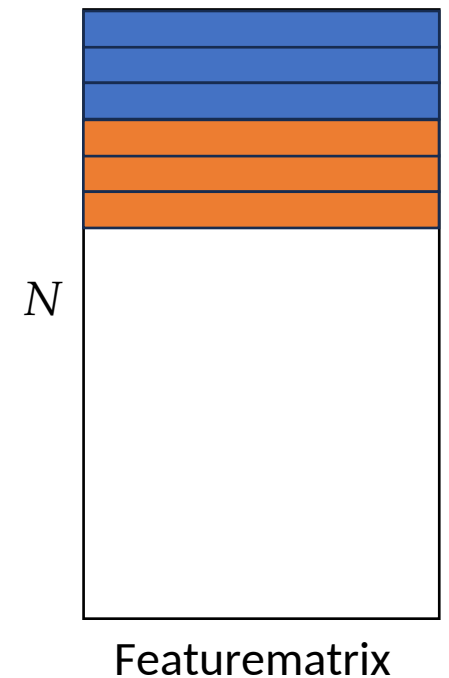
$$\mathbf{w}^{[1,1]} = \mathbf{w}^{[1,0]} - \alpha^{[0]} \sum_{j=1}^{10} \nabla_{\mathbf{w}} \left((y_1 - (w_1 x_{j,1} + w_2 x_{j,2}))^2 \right)$$

3. Setze $i = 2$

Nächste Iterierte durch Gradienten der **2. Minibatch**:

$$\mathbf{w}^{[1,2]} = \mathbf{w}^{[1,1]} - \alpha^{[0]} \sum_{j=11}^{20} \nabla_{\mathbf{w}} \left((y_1 - (w_1 x_{j,1} + w_2 x_{j,2}))^2 \right)$$

4. ...



Beispiel Minibatch SGD

...

11. Setze $i = 10$

Nächste Iterierte durch Gradienten der 10. Minibatch Trainingsbeispiels:

$$\mathbf{w}^{[1,100]} = \mathbf{w}^{[1,99]} - \alpha^{[0]} \sum_{j=91}^{100} \nabla_{\mathbf{w}} \left(\left(y_1 - (w_1 x_{j,1} + w_2 x_{j,2}) \right)^2 \right)_N$$

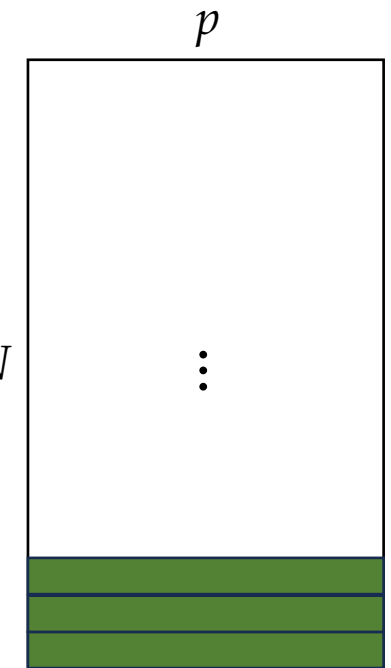
12. Setze $\mathbf{w}^{[1]} = \mathbf{w}^{[1,10]}$

13. Zweite Epoche: Wähle Lernrate $\alpha^{[1]}$, setze $\mathbf{w}^{[2,0]} = \mathbf{w}^{[1]}$

14. Setze $i = 1$

15. (Iteriere wieder über die 10 Minibatches/Blöcke in der Featurematrix)

usw.



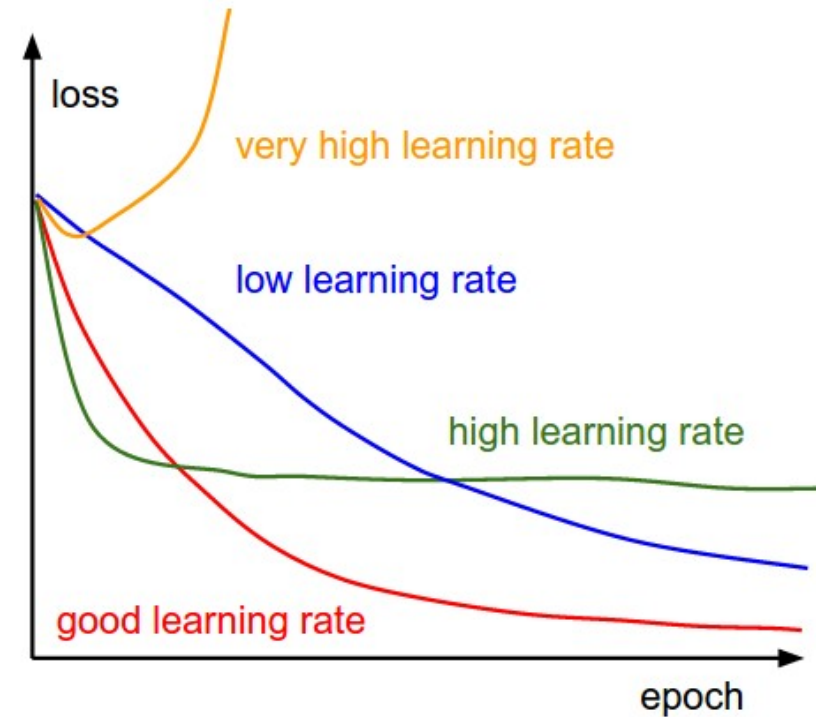
Featurematrix

Praktische Aspekte

- Batchgröße m : Hyperparameter (oft: Zweier-Potenzen)
- Rechenzeit pro Iteration unabhängig von der Größe des gesamten Trainingsdatensatzes (ist bei Batch GD nicht der Fall)
- Bei großen Trainingsdatensätzen relevant: Der Trainingsdatensatz (Featurematrix) muss nie komplett im Hauptspeicher vorliegen, es genügt, die Daten Minibatch-weise einzulesen. Beispiel: Vorhersagen auf Bildern, 128x128 Pixel:
 - Featurematrix: $10,000 \text{ Bilder} \times (128 \times 128 \times 3 \text{ Kanäle}) \times 8 \text{ bytes} \approx 4GB$
- Minibatch SGD ist Standard bei aktuellen Optimierungsverfahren

Wahl der Lernrate

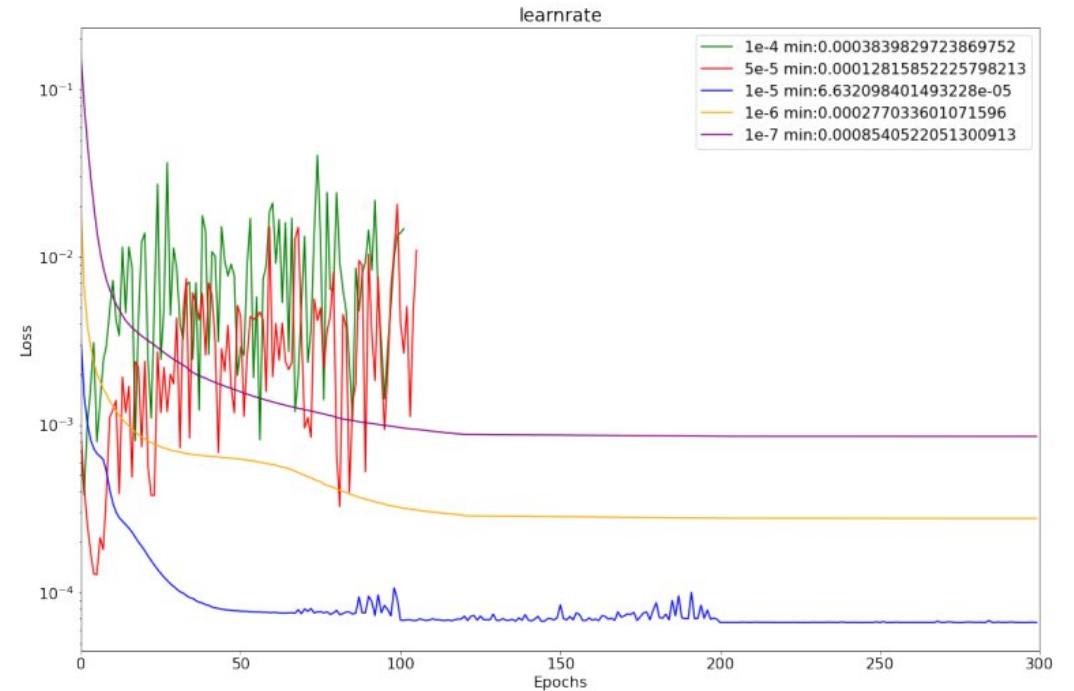
- Lernrate $\alpha^{[k]}$ ist Hyperparameter mit oft großem Einfluss auf den Verlauf des Trainings.
- Allgemein: Verlauf der Verlustfunktion über die Epochen plotten („Losskurve“) ist hilfreiches Werkzeug.
 - X-Achse: Anzahl Epochen
 - Y-Achse: Verlust auf Trainingsdaten nach der Epoche
- Wahl der Lernrate: Trainingsläufe für verschiedene Lernraten auf kleiner Teilmenge der Trainingsdaten und Loss-Kurve beobachten.



Wahl der Lernrate: Faustregel

Einfaches Rezept („Learning schedule“):

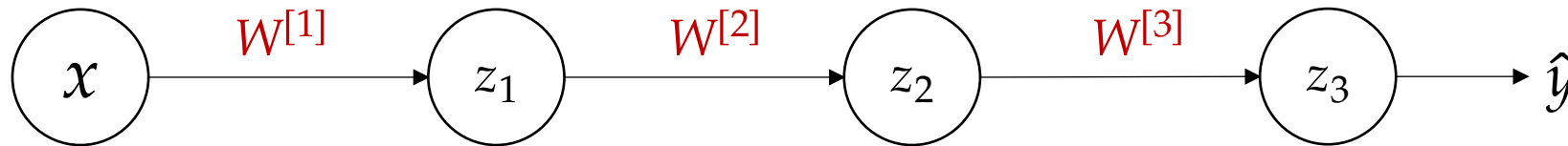
1. Finde größte Lernrate, so dass SGD nicht divergiert (Genauigkeit: Faktor 10)
2. Starte SGD mit konstanter Lernrate bis Plateau in Losskurve erreicht ist
3. Reduziere Lernrate um konstanten Faktor (z.B. 10)
4. Wiederhole 2. und 3. bis Konvergenz



6.3 Berechnung der Gradienten (Backpropagation)

Ableitungen von neuronalen Netzen

- Für das einfache Beispiel (lineare Regression) haben wir den Gradienten der Verlustfunktion per Hand berechnet.
- Beispiel für ein einfaches neuronales Netz (Ann.: Alle $W^{[i]}$ skalar, Regressionsproblem):



In Formeln: $\hat{y} = W^{[3]} \sigma(W^{[2]} \sigma(W^{[1]}x + b^{[1]}) + b^{[2]}) + b^{[3]}$

- Gesucht: Ableitung des MSE: $\nabla L(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, b^{[3]})$
- $$\begin{aligned}
 &= \nabla \left(\sum_{i=1}^N (y_i - \hat{y}(x_i; \mathbf{w}))^2 \right) = \sum_{i=1}^N \nabla (y_i - \hat{y}(x_i; \mathbf{w}))^2 = 2 \sum_{i=1}^N (y_i - \hat{y}(x_i; \mathbf{w})) \left(-\frac{\partial \hat{y}}{\partial \mathbf{w}} \right) \\
 &= -2 \sum_{i=1}^N (y_i - \hat{y}(x_i; \mathbf{w})) \left(\frac{\partial \hat{y}}{\partial W^{[1]}}, \frac{\partial \hat{y}}{\partial b^{[1]}}, \frac{\partial \hat{y}}{\partial W^{[2]}}, \frac{\partial \hat{y}}{\partial b^{[2]}}, \frac{\partial \hat{y}}{\partial W^{[3]}}, \frac{\partial \hat{y}}{\partial b^{[3]}} \right) = \dots
 \end{aligned}$$

- Das ist prinzipiell möglich (Kettenregel!) aber selbst für einfache Beispiele nicht praktikabel.

Automatische Differentiation

Uns fehlt noch eine Methode, um **automatisch und effizient** Gradienten vom Output neuronaler Netze (oder auch beliebigen anderen komplizierten Funktionen) nach den Gewichten zu berechnen:

Der **Rückwärtsmodus der automatischen Differentiation**. Im Kontext von neuronalen Netzen wird die Technik als **Backpropagation** bezeichnet.

Wir betrachten wieder zunächst einfache Funktionen (d.h. keine neuronalen Netze) um die Konzepte zu verdeutlichen.

Grundlagen: Berechnungsgraphen

Jede in Quellcode implementierte mathematische Funktion kann als Abfolge elementarer Operationen aufgefasst werden.

Beispiel: $f(\mathbf{x}) = (x_1 x_2 \sin x_3 + e^{x_1 x_2})/x_3$

$$x_4 = x_1 * x_2,$$

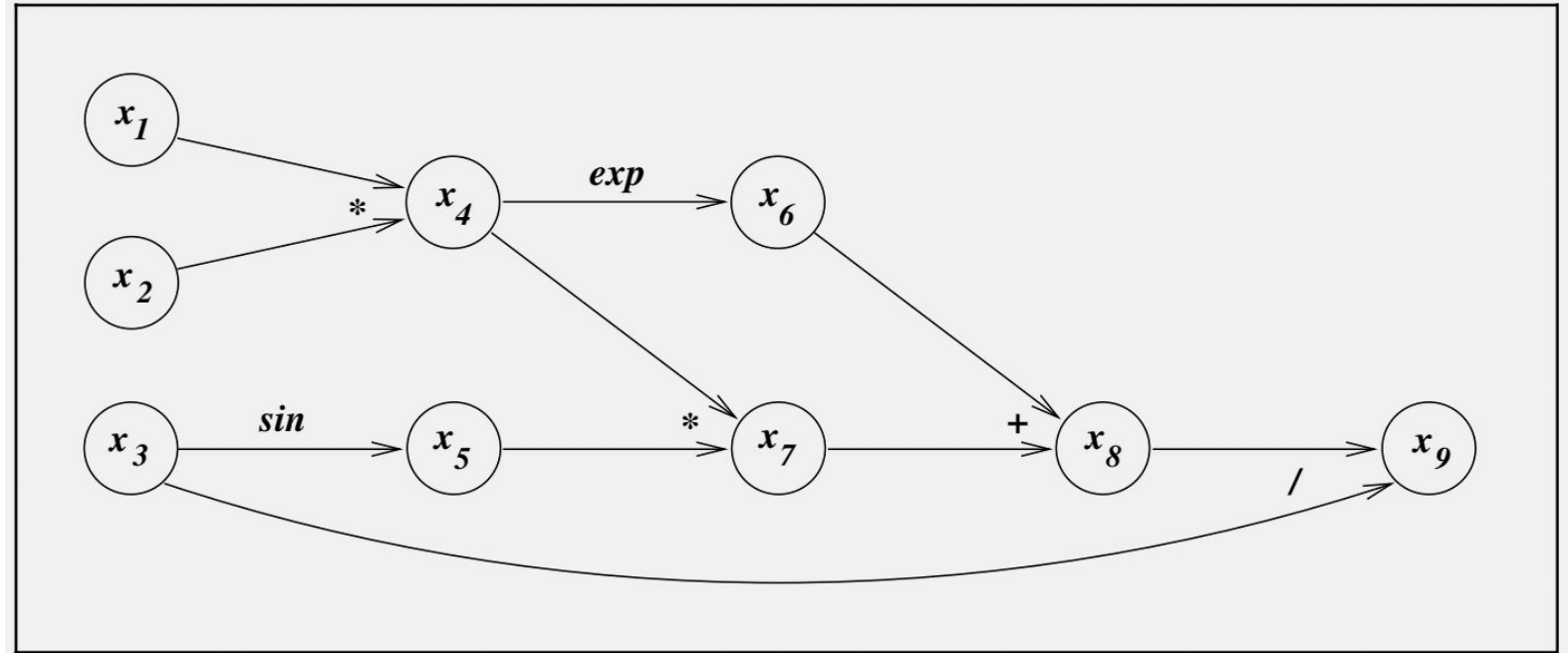
$$x_5 = \sin x_3,$$

$$x_6 = e^{x_4},$$

$$x_7 = x_4 * x_5,$$

$$x_8 = x_6 + x_7,$$

$$x_9 = x_8 / x_3.$$



$x_4, \dots, x_9$ sind **Zwischenvariablen** (im Gegensatz zu den **unabhängigen Variablen** x_1, x_2, x_3). x_9 enthält den Funktionswert $f(\mathbf{x})$.

Grundlagen: Berechnungsgraphen

Wichtig: die Zwischenvariablen $x_4, \dots, x_9$ müssen nicht explizit als Variablen im Code eingeführt werden. Sie repräsentieren vielmehr die logische Abfolge der Berechnungsschritte. Tools zur automatischen Ableitungserzeugung erzeugen diesen Berechnungsgraphen automatisch (z.B. PyTorch).

$$x_4 = x_1 * x_2,$$

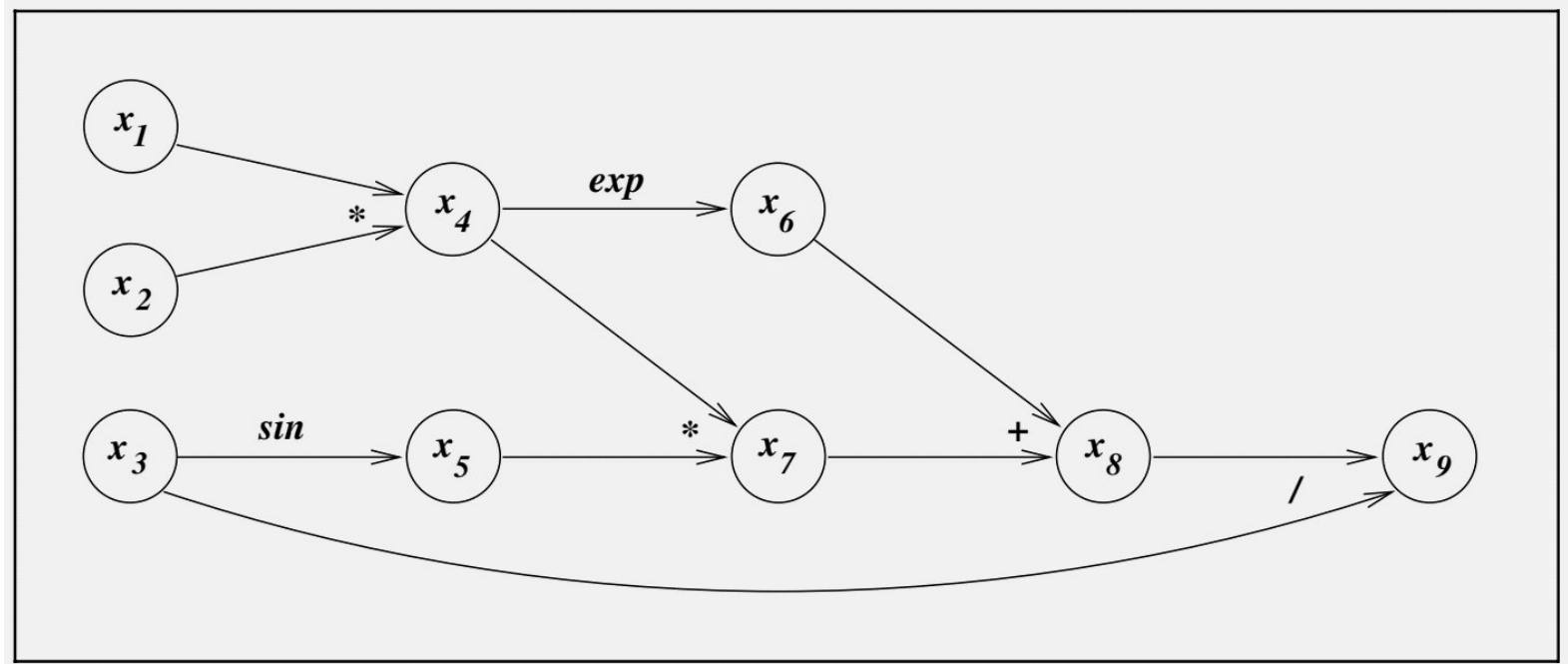
$$x_5 = \sin x_3,$$

$$x_6 = e^{x_4},$$

$$x_7 = x_4 * x_5,$$

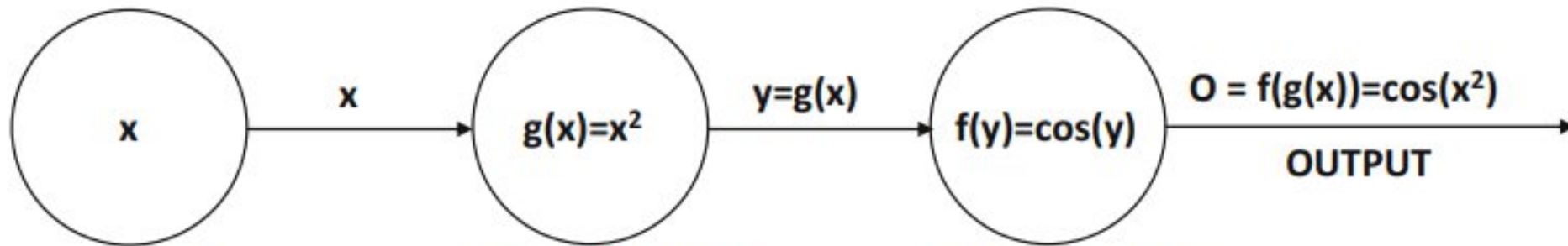
$$x_8 = x_6 + x_7,$$

$$x_9 = x_8 / x_3.$$



Beispiel: $f(\mathbf{x}) = (x_1 x_2 \sin x_3 + e^{x_1 x_2}) / x_3$

Berechnungsgraphen beschreiben Verkettung von Funktionen!



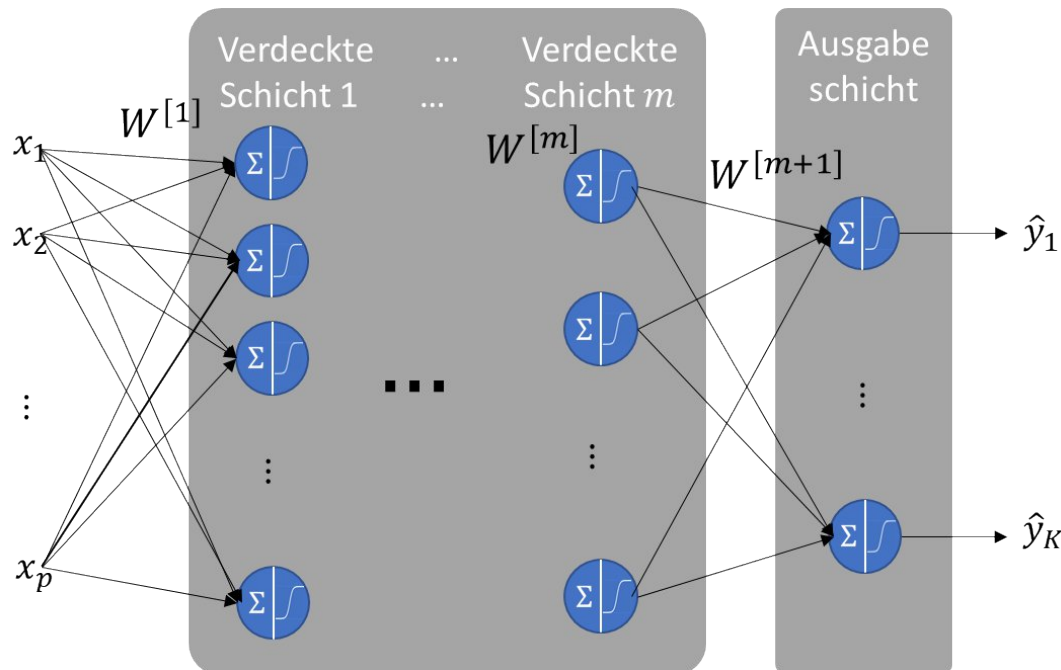
$$f(x) = \cos x, \quad g(x) = x^2.$$

Graph beschreibt die Verkettung $f(g(x)) = \cos x^2$

Verkettungen von Funktionen werden schnell unhandlich. Bsp:

$$f(x) = g(x) = \frac{1}{1 + \exp(-x)}$$
$$\Rightarrow f(g(x)) = \frac{1}{1 + \exp\left(-\frac{1}{1 + \exp(-x)}\right)}$$

Neuronale Netze sind Funktionen, die als Berechnungsgraphen dargestellt werden.



$$\mathbf{a}^{[0]} = \mathbf{x}$$

Für $k = 1, \dots, m$:

$$\mathbf{a}^{[k]} = \mathbf{g}(\mathbf{W}^{[k]} \mathbf{a}^{[k-1]} + \mathbf{b}^{[k]})$$

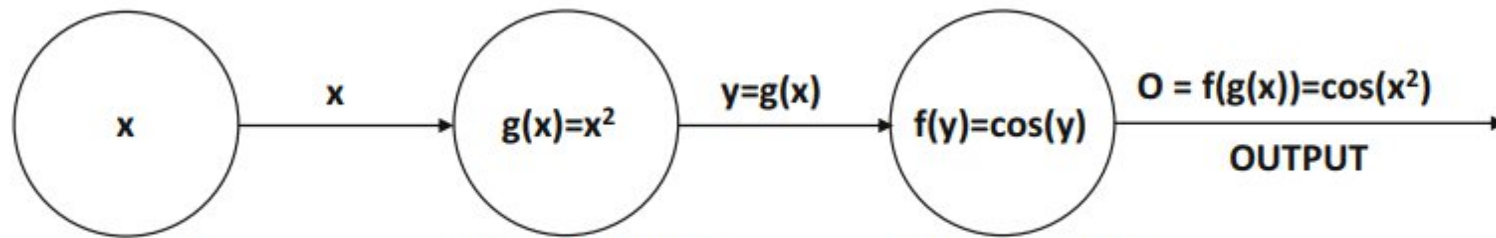
$$\hat{\mathbf{y}} = f(\mathbf{W}^{[m+1]} \mathbf{a}^{[m]} + \mathbf{b}^{[m+1]}),$$

Dadurch ergibt sich als Output des Netzes die rekursive Verkettung:

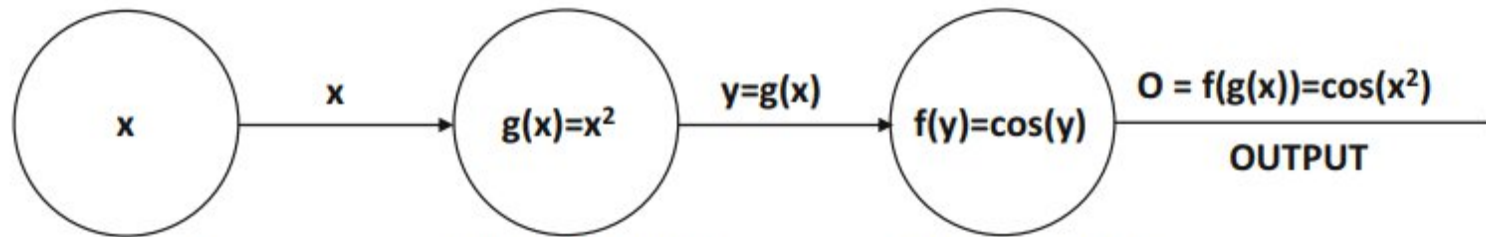
$$\hat{\mathbf{y}} = f(\mathbf{W}^{[m+1]} \mathbf{g}(\mathbf{W}^{[m]} \mathbf{g}(\mathbf{W}^{[m-1]} \mathbf{g}(\mathbf{W}^{[m-2]} (\dots))))$$

Ableitungen auf Berechnungsgraphen

- Zusammenfassend: Berechnungsgraphen sind eine Darstellungsform von Funktionen
- Für unser Ziel, Ableitungen von Funktionen zu berechnen benutzen wir die Darstellung einer Funktion als Berechnungsgraph.
- Da ein Berechnungsgraph die **Verkettung** von Funktionen darstellt, benutzt man dazu die ... Kettenregel!



Wiederholung: Univariate Kettenregel

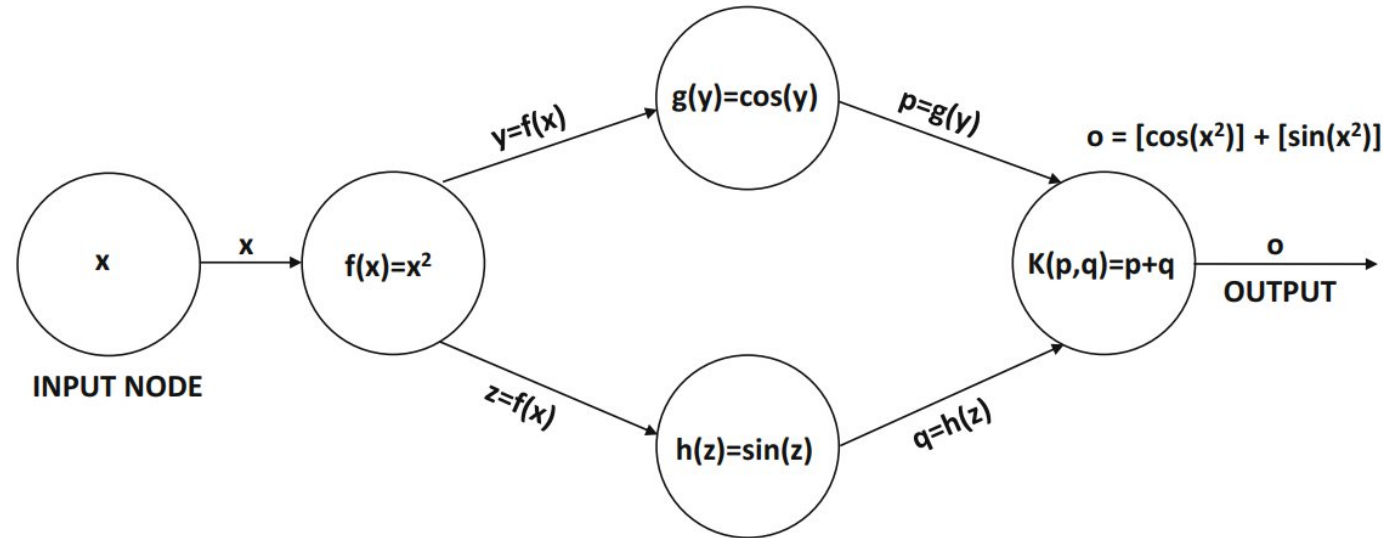


Beispiel: Ableitung von $f(g(x))$ wobei $f(y) = \cos y$, $g(x) = x^2$, (also insgesamt $\cos(x^2)$):

$$\frac{\partial f(g(x))}{\partial x} = \underbrace{\frac{\partial f(g(x))}{\partial g(x)}}_{-\sin g(x)} \cdot \underbrace{\frac{\partial g(x)}{\partial x}}_{2x} = -\sin(x^2) \cdot 2x$$

„Äußere Ableitung x innere Ableitung“ oder: Jeden Knoten mit seiner Ableitung (bzgl. des Inputs) annotieren, dann Knoten entlang des Pfades multiplizieren!

Beispiel: $\sin(x^2) + \cos(x^2)$

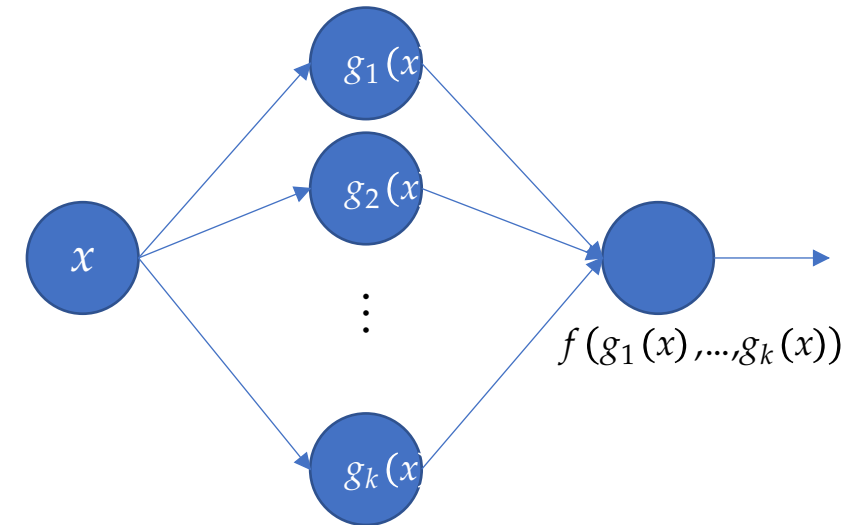


$$\begin{aligned}\frac{\partial o}{\partial x} &= \underbrace{\frac{\partial K(p, q)}{\partial p}}_1 \cdot \underbrace{g'(y)}_{-\sin(y)} \cdot \underbrace{f'(x)}_{2x} + \underbrace{\frac{\partial K(p, q)}{\partial q}}_1 \cdot \underbrace{h'(z)}_{\cos(z)} \cdot \underbrace{f'(x)}_{2x} \\ &= -2x \cdot \sin(y) + 2x \cdot \cos(z) \\ &= -2x \cdot \sin(x^2) + 2x \cdot \cos(x^2)\end{aligned}$$

- Zwei Pfade von x nach o
- Die Ableitung $f'(x) = 2x$ wurde zweimal ausgewertet!

Multivariate Kettenregel: Spezialfall

- Wenn man eine univariate(!) Funktion als Berechnungsgraph darstellen möchte, geht das i.A. nicht als einfacher Pfad.
- Eine Eingabevariable könnte z.B. an k unabhängige Knoten „fließen“, die die Funktionen $g_1(x), g_2(x), \dots, g_k(x)$ berechnen.
- Diese könnten wiederum ihr Ergebnis an die Ausgabefunktion f (mit k Argumenten) weiterleiten, welche dann $f(g_1(x), \dots, g_k(x))$ berechnet.



Um f nach x abzuleiten (das ist ein Skalar!) braucht man die multivariate Kettenregel:

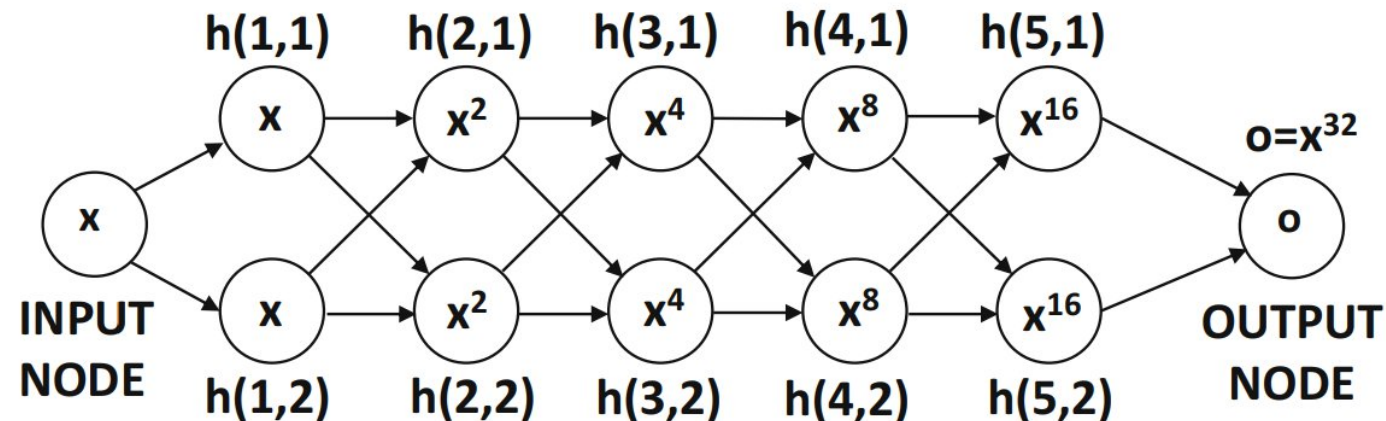
$$\frac{\partial f(g_1(x), \dots, g_k(x))}{\partial x} = \sum_{i=1}^k \frac{\partial f(g_1(x), \dots, g_k(x))}{\partial g_i(x)} \cdot \frac{\partial g_i(x)}{\partial x}$$

Auf Berechnungsgraphen kann man die multivariate Kettenregel nun so interpretieren:

Die Ableitung der Variable im Output Knoten t nach der Variable in einem Eingangsknoten s ist die **Summe der Ableitungen entlang aller möglichen Pfade zwischen s und t** . Die Ableitungen entlang der Pfade können mit der unvariaten Kettenregel als Produkte der (lokalen) Ableitungen an jedem Knoten berechnet werden.

Nochmal: Pfad-Interpretation der Kettenregel

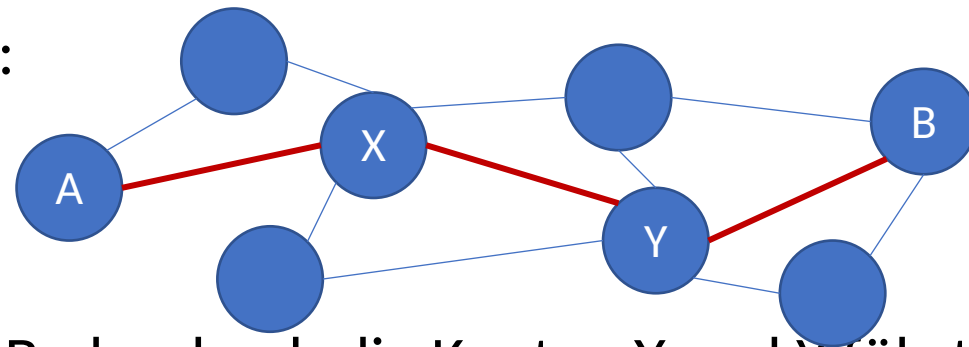
- Ableitung Output Knoten t nach der Eingangsknoten s = Multiplikation der lokalen Ableitungen entlang der Pfade und Summe über **alle möglichen** Pfade.
- Bsp: 32 Pfade von Input nach Output. Jede Node liegt auf 16 Pfaden (wird also 16 mal ausgewertet)



- Bsp: Fully connected network mit 3 Layern und 100 Knoten in jedem Layer: 1,000,000 Pfade!
- —> Die Pfad Interpretation ist eine interessante Einsicht, hilft aber zunächst nicht weiter bei der Berechnung.

Einschub: Dynamische Programmierung

- Optimalitätsprinzip (Bellman) eigentlich: „Lösung eines Optimierungsproblems setzt sich aus optimalen Teillösungen zusammen.“
- Beispiel (kürzeste Wege in Graphen):



Ein kürzester Weg P zwischen A und B , der durch die Knoten X und Y führt, muss auch zwischen X und Y einen kürzesten Weg zwischen diesen beiden Knoten verwenden (und auch zwischen Y und B , Y und A , usw.).

- Für unseren Fall $\approx$ „Wenn man die Summe über alle Pfade zwischen irgendeinem Knoten y und dem Output ($=\partial o/\partial y$) einmal berechnet hat, können alle Pfade, die durch y führen, auf diese einmal berechnete Ableitung zurückgreifen.“

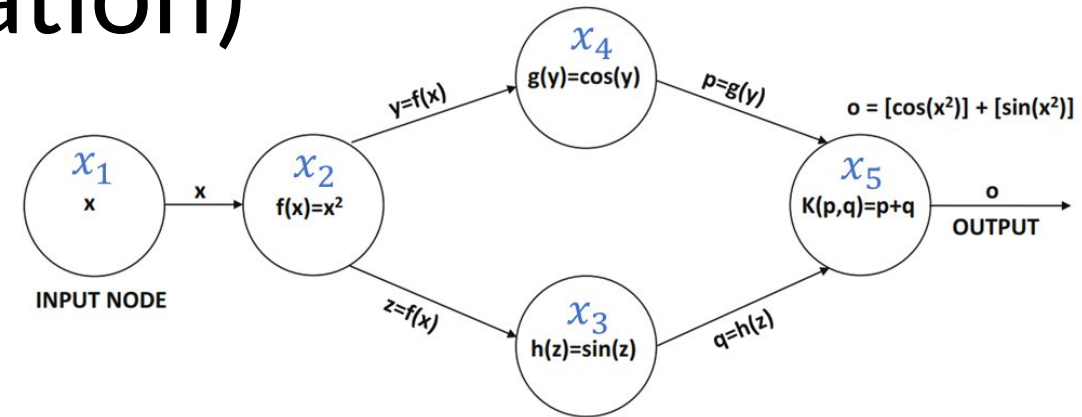
DP vs. Backpropagation vs. rückwärts AD

- Mittels dynamischer Programmierung bekommt man in einem Durchlauf die Ableitung des Output Knotens nach *allen* Variablen (Knoten).
- Man nennt dies den „Rückwärtsmodus der automatischen Differentiation“.
- Insbesondere im Kontext neuronaler Netze wird der Algorithmus als Backpropagation (of errors) bezeichnet (eigtl. ein Spezialfall des Rückwärtsmodus).
- Im Laufe des Verfahrens wird jeder Knoten und jede Kante des Graphen (zweimal) bearbeitet, das Verfahren unterliegt also *keinem* exponentiellen Wachstum!

Algorithmus (Backpropagation)

Gegeben:

- Gerichteter, azyklischer Berechnungsgraph mit Knoten x_i
- x : Wert des Arguments
- f : Variable im Output Knoten (Funktion, deren Ableitung berechnet werden soll)
- Für jedes Kind j des Knoten i bezeichne $\partial x_j / \partial x_i$ die („lokale“) Ableitung (=wie ändert sich x_j , wenn sich x_i ändert, unabhängig davon was vor i und nach j geschieht)

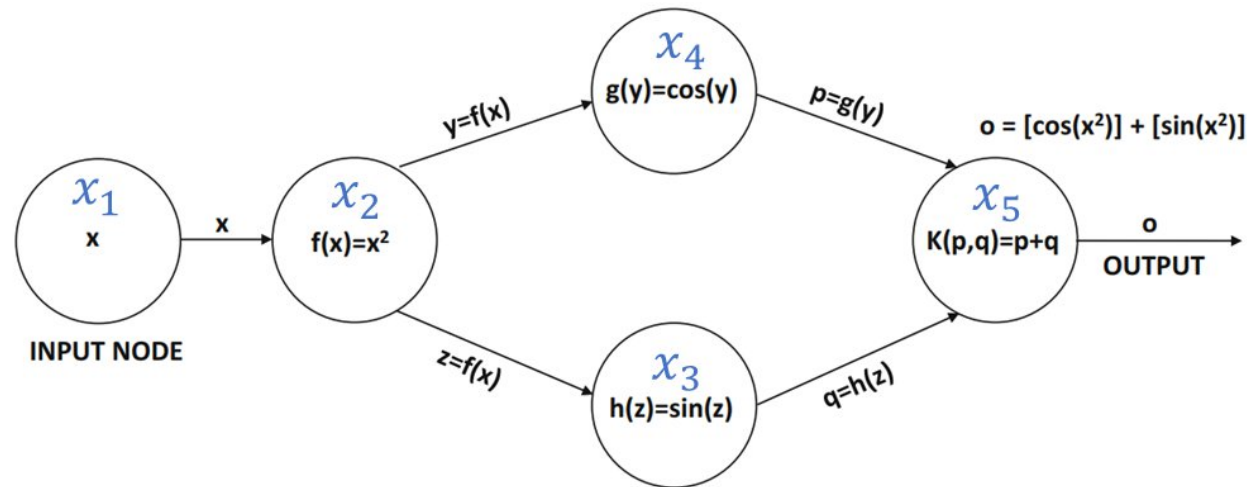


Gesucht:

- Funktionswerte an allen Knoten x_i (insbes. $f(x)$)
- Alle partiellen Ableitungen $\frac{\partial f}{\partial x_i}$ (insbes. $\frac{\partial f}{\partial x}$)

Backpropagation: Vorgehen

1. **Vorwärtsphase:** Die Funktion f wird ausgewertet. Dabei werden alle Knoten besucht und an jedem Knoten wird der Wert x_i gespeichert.
2. **Rückwärtsphase:** Der Graph wird rückwärts, d.h. ausgehend von f traversiert, um die Werte der Ableitungen zu berechnen. Dabei wird auf die Werte aus der Vorwärtsphase zurückgegriffen.



Backpropagation: Rückwärtsphase

- Für jeden Knoten führen wir eine skalare Variable $\bar{x}_i$ ein (sog. **adjungierte Variable**)
- Adjungierte Variable am Knoten f : $\bar{x}_N$
- Am Ende des Algorithmus soll gelten $\bar{x}_i = \frac{\partial f}{\partial x_i}$.

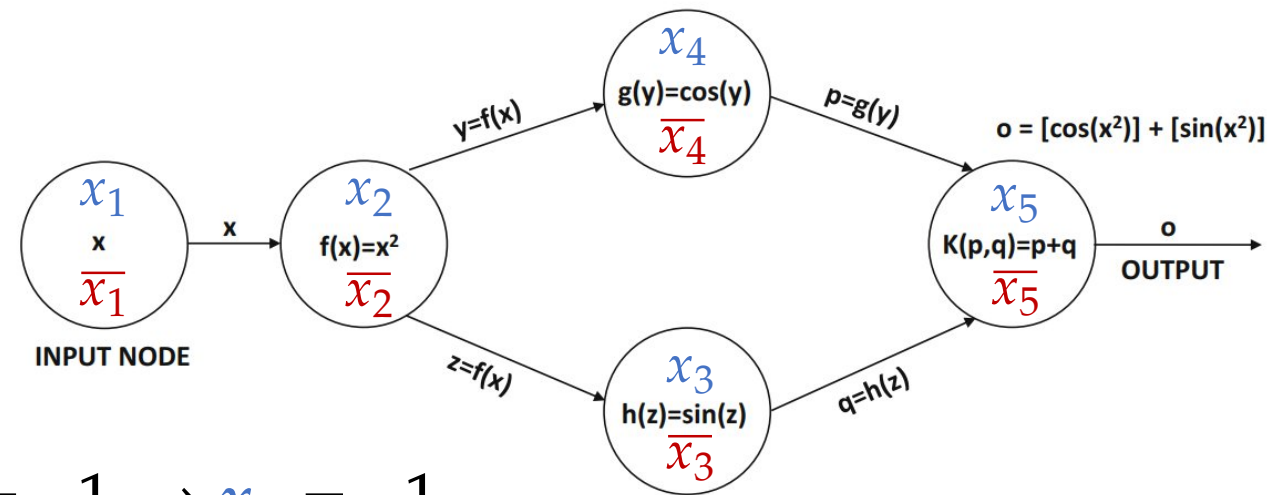
Schritte Rückwärtsphase:

1. Start: Initialisiere $\bar{x}_N = \frac{\partial f}{\partial f} = 1$
2. Solange noch Knoten vorhanden sind, die noch nicht ausgewählt wurden: Wähle unbearbeiteten Knoten i , von dem alle Werte $\bar{x}_j$ der Kinder bekannt sind
3. Setze

$$\bar{x}_i = \sum_{j \text{ Kind von } i} \bar{x}_j \cdot \frac{\partial x_j}{\partial x_i} = \sum_{j \text{ Kind von } i} \frac{\partial f}{\partial x_j} \cdot \frac{\partial x_j}{\partial x_i}$$

Das ist die Kettenregel!!

Beispiel für $x = \sqrt{\pi}$



Vorwärtsphase:

$$x_1 = \sqrt{\pi} \rightarrow x_2 = \pi \rightarrow x_3 = 0 \rightarrow x_4 = -1 \rightarrow x_5 = -1$$

Rückwärtsphase:

1. Setze $\overline{x_5} = 1$
2. Wähle $\overline{x_4}$. Setze $\overline{x_4} = \overline{x_5} \cdot \frac{\partial x_5}{\partial x_4} = 1$
3. Wähle $\overline{x_3}$. Setze $\overline{x_3} = \overline{x_5} \cdot \frac{\partial x_5}{\partial x_3} = 1$
4. Wähle $\overline{x_2}$. Setze $\overline{x_2} = \overline{x_4} \cdot \frac{\partial x_4}{\partial x_2} + \overline{x_3} \cdot \frac{\partial x_3}{\partial x_2} = 1 \cdot (-\sin x_2) + 1 \cdot \cos x_2 = -1$
5. Wähle $\overline{x_1}$. Setze $\overline{x_1} = \overline{x_2} \cdot \frac{\partial x_2}{\partial x_1} = -1 \cdot 2x_1 = -2\sqrt{\pi}$

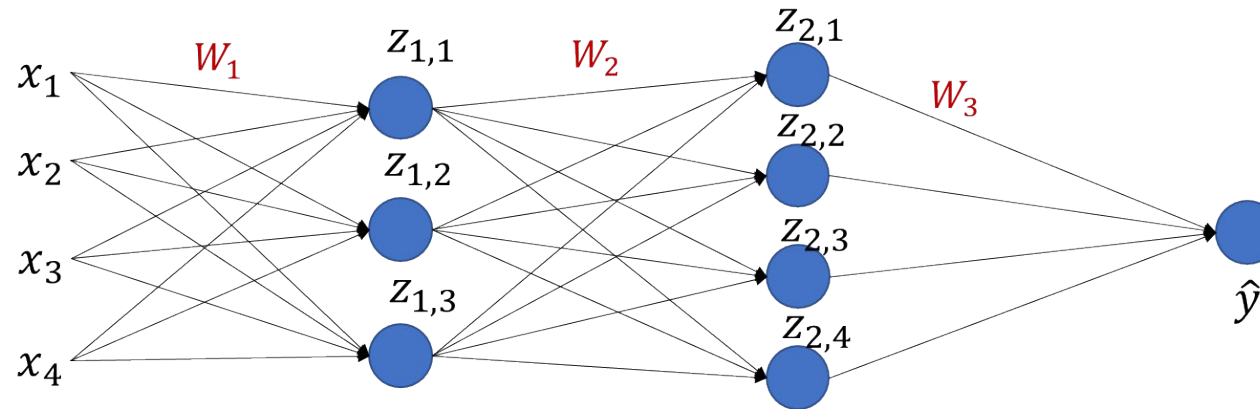
Zusammenfassung Backpropagation (I)

- Berechnungsgraphen sind eine alternative Form der Darstellung von Funktionen als gerichtete Graphen.
- Jeder Knoten im Graphen repräsentiert eine differenzierbare Operation (z.B. $+$, $*$, $\sin$, $\exp$, ...), Kanten repräsentieren die Verkettung dieser Operationen.
- Der Vorteil dieser Darstellung ist, dass **Ableitungen** mittels effizienter Anwendung der Kettenregel automatisch numerisch ausgewertet werden können.
- Die Effizienz kommt daher, dass der Berechnungsgraph (rückwärts) in einer bestimmten Abfolge traversiert wird (dynamische Programmierung).
- Backpropagation bezeichnet also die effiziente Anwendung der Kettenregel auf (große) Berechnungsgraphen.

Backpropagation: Anwendung auf neuronale Netze

Ableitungserzeugung bei neuronalen Netzen

- Fast das gleiche wie ein Berechnungsgraph!

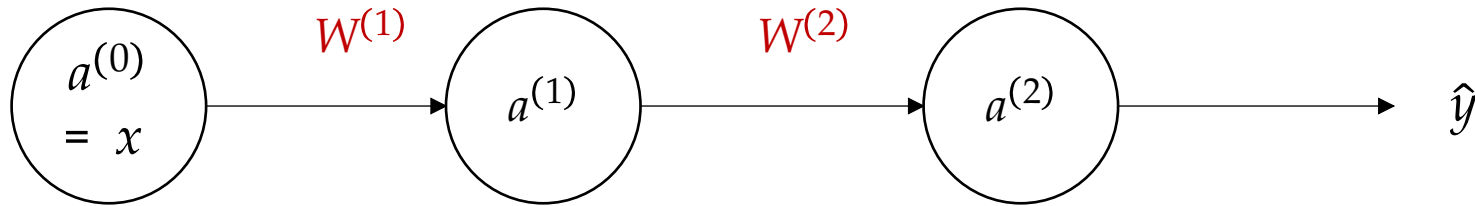


- Zwei Unterschiede:
 1. Künstliches Neuron ist keine „elementare Operation“ (wie im Berechnungsgraph)
 2. SGD braucht Ableitung nach Kanten **W**, nicht nach Eingangsgrößen x

Einfaches Beispiel

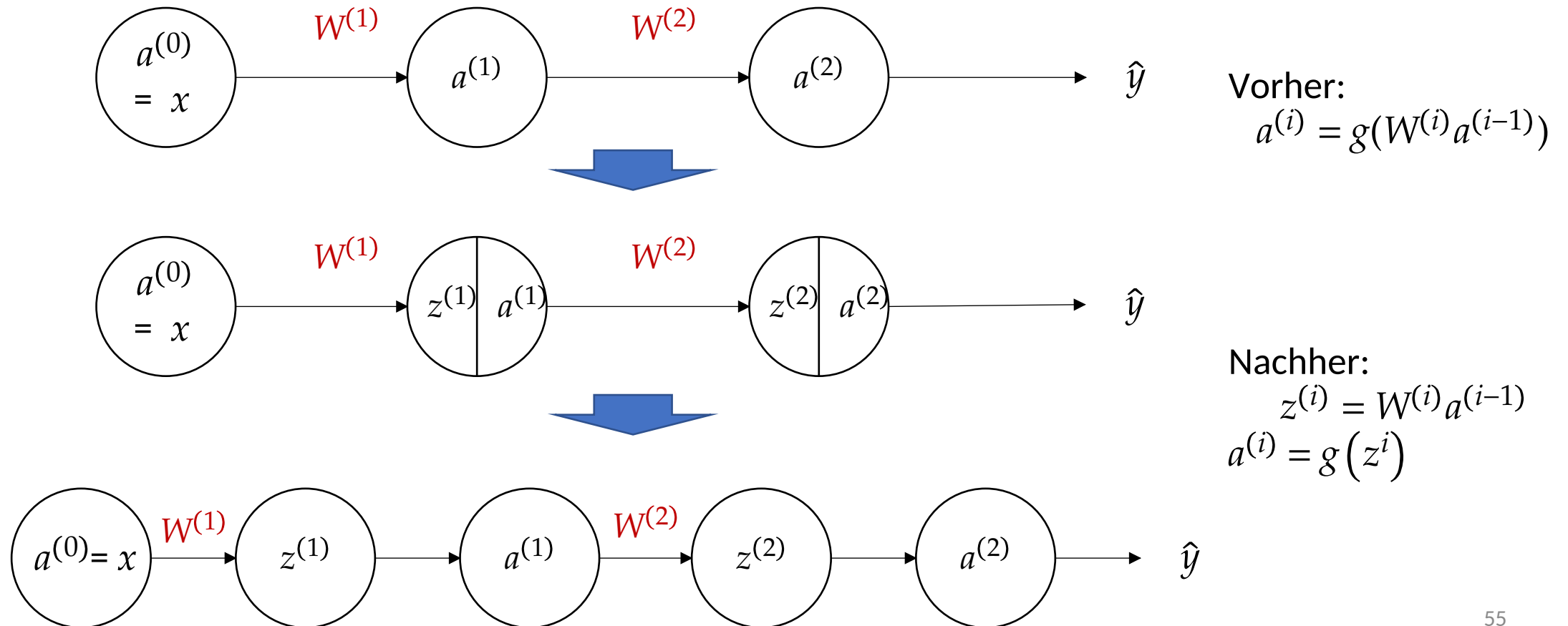
Berechnung: $a^{(i)} = g(W^{(i)} a^{(i-1)})$ mit Aktivierungsfunktion g

Annahme zunächst: Alle W sind Skalare.



Schritt 1: Künstliches Neuron als zwei Funktionen

Betrachte ein künstliches Neuron als *zwei* Knoten



Schritt 2:

Ableitung der Knoten nach Vorgängern

Partielle Ableitung der beiden Knotentypen

Aus der Definition:

$$z^{(i)} = W^{(i)} a^{(i-1)}$$

$$a^{(i)} = g(z^{(i)})$$

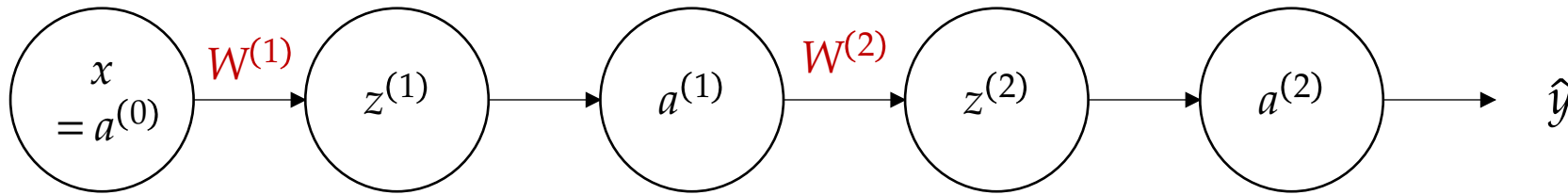
berechnet man die Ableitungen

$$\frac{\partial z^{(i)}}{\partial a^{(i-1)}} = W^{(i)} \text{ (Ableitung lineare Funktion)}$$

$$\frac{\partial a^{(i)}}{\partial z^{(i)}} = g'(z^{(i)}) \text{ (Ableitung der Aktivierungsfunktion, z.B. log. Fkt., tanh, ReLU)}$$

Schritt 3:

Normale Kettenregel



Da wir die Multiplikation mit W nun innerhalb der z -Knoten verarbeiten, gilt weiterhin: Kettenregel=„Multiplikation entlang des Pfades“:

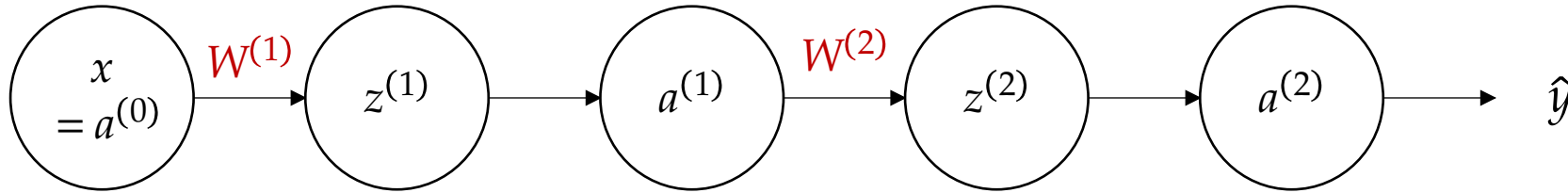
$$\frac{\partial \hat{y}}{\partial a^{(0)}} = \frac{\partial \hat{y}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial a^{(0)}}$$

bzw.:

$$\frac{\partial \hat{y}}{\partial z^{(1)}} = \frac{\partial \hat{y}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}}$$

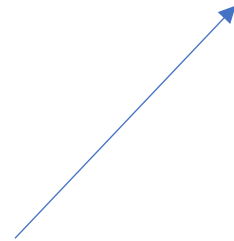
Schritt 4:

Ableitung nach W



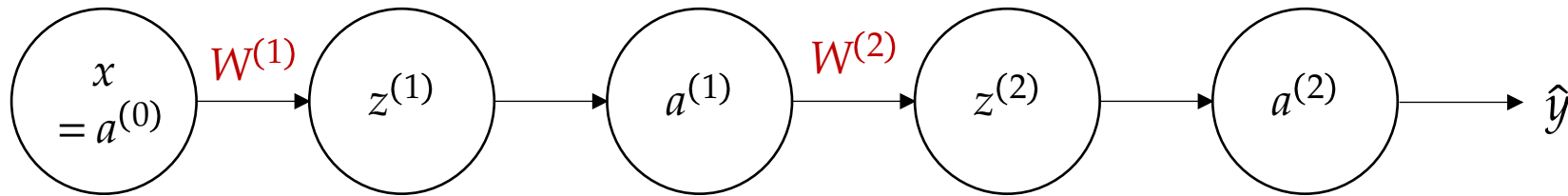
- Man kann also zunächst vorgehen wie bei der „normalen“ Rückwärtsableitung und die Ableitung nach allen Knoten $a^{(i)}$ und $z^{(i)}$ berechnen.
- Anschließend kann man die Ableitung nach allen W_i wie folgt berechnen:

$$\frac{\partial \hat{y}}{\partial W^{(i)}} = \frac{\partial \hat{y}}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial W^{(i)}} = \frac{\partial \hat{y}}{\partial z^{(i)}} \cdot a^{(i-1)}$$



Gradient bis zum Knoten unmittelbar nach W :
Diese Ableitung wird im „normalen“ Rückwärts-Lauf berechnet

Ableitungserzeugung bei neuronalen Netzen: Beispiel (1/3)



$g(z) = \frac{1}{1+e^{-z}}$. Es gilt: $\sigma'(z) = \sigma(z)(1-\sigma(z))$ (siehe Fußnote).

Beispiel: Backpropagation abgebildetes NN für Eingabewert $x = 1$ und Gewichte $W^{(1)} = 2$, $W^{(2)} = -1$.

1. Schritt: Vorwärtslauf (Werte an allen Knoten):

$$a^{(0)} = x = 1$$

$$z^{(1)} = W^{(1)} a^{(0)} = 2 \cdot 1 = 2$$

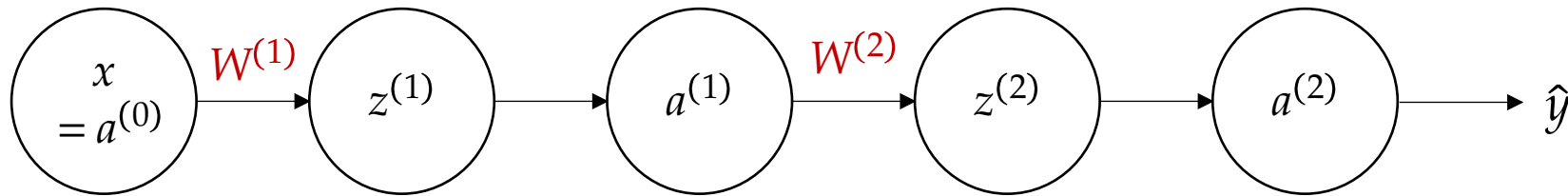
$$a^{(1)} = g(z^{(1)}) = \frac{1}{1+e^{-2}} = 0.88$$

$$z^{(2)} = W^{(2)} a^{(1)} = -1 \cdot 0.88 = -0.88$$

$$a^{(2)} = g(z^{(2)}) = \frac{1}{1+e^{0.88}} = 0.29$$

$$\hat{y} = a^{(2)} = 0.29$$

Ableitungserzeugung bei neuronalen Netzen: Beispiel (2/3)



$$g(z) = \frac{1}{1+e^{-z}}. \text{ Es gilt: } \sigma'(z) = \sigma(z)(1-\sigma(z)).$$

Beispiel: Backpropagation abgebildetes NN für Eingabewert $x = 1$ und Gewichte $W^{(1)} = 2$, $W^{(2)} = -1$.

2. Schritt: Rückwärtslauf (zunächst alle Ableitungen nach z und a).

Notation für die Adjungierten wie zuvor: $\bar{z}_i = \partial \hat{y} / \partial z_i$:

$$\bar{a}^{(2)} = 1$$

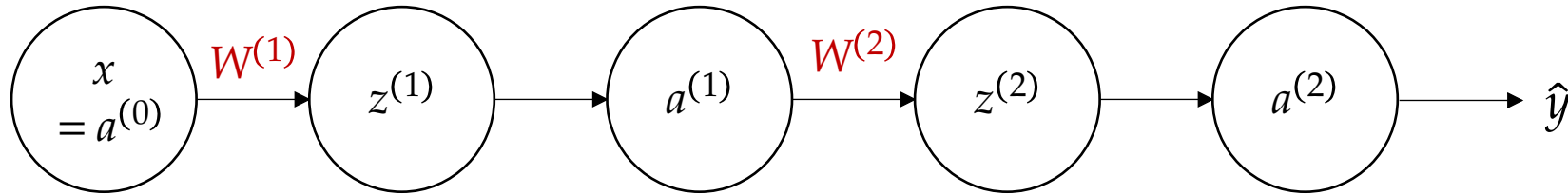
$$\bar{z}^{(2)} = \bar{a}^{(2)} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} = 1 \cdot g'(z^{(2)}) = 1 \cdot g'(-0.88) = 0.21$$

$$\bar{a}^{(1)} = \bar{z}^{(2)} \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} = 0.21 \cdot W^{(2)} = 0.21 \cdot (-1) = -0.21$$

$$\bar{z}^{(1)} = \bar{a}^{(1)} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} = -0.21 \cdot g'(z^{(1)}) = -0.21 \cdot g'(2) = -0.02$$

$$\bar{a}^{(0)} = \bar{z}^{(1)} \cdot \frac{\partial z^{(1)}}{\partial a^{(0)}} = -0.02 \cdot W^{(1)} = -0.02 \cdot 2 = -0.04$$

Ableitungserzeugung bei neuronalen Netzen: Beispiel (3/3)



$g(z) = \frac{1}{1+e^{-z}}$. Es gilt: $\sigma'(z) = \sigma(z)(1-\sigma(z))$.

Beispiel: Backpropagation abgebildetes NN für Eingabewert $x = 1$ und Gewichte $W^{(1)} = 2$, $W^{(2)} = -1$.

3. Schritt: „Nachdifferenzieren“ nach W , um die gewünschten Ableitungen zu erhalten:

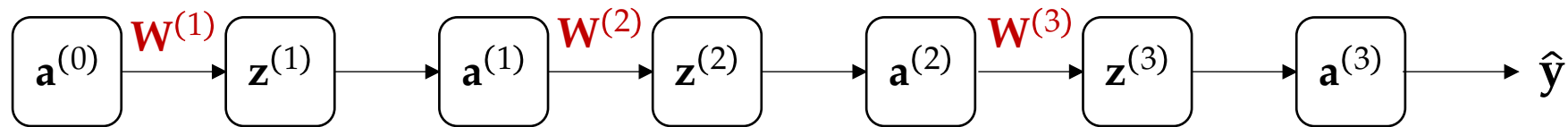
$$\bar{W}^{(2)} = \bar{z}^{(2)} \cdot \frac{\partial z^{(2)}}{\partial W^{(2)}} = \bar{z}^{(2)} \cdot a^{(1)} = 0.21 \cdot 0.88 = 0.18$$

$$\bar{W}^{(1)} = \bar{z}^{(1)} \cdot \frac{\partial z^{(1)}}{\partial W^{(1)}} = \bar{z}^{(1)} \cdot a^{(0)} = -0.02 \cdot (-0.04) = 0.0008$$

Allgemeiner Fall ($\mathbf{W}^{(i)}$ sind Matrizen)

Für ein **allgemeines Feedforward-Netzwerk** fehlt noch:

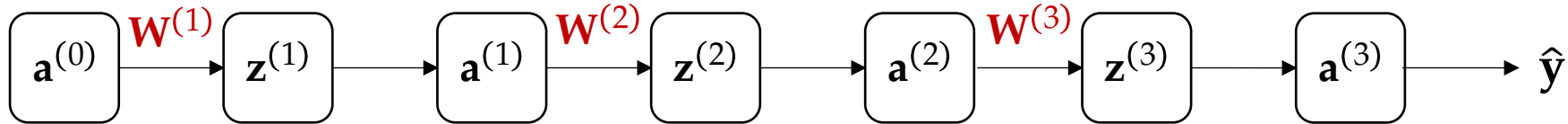
1. Verlustfunktion
2. Matrix-wertige Variablen, d.h. $\mathbf{W}^{(i)}$ sind Matrizen:



Lösung für 1.:

Verlustfunktion ist auf $\hat{\mathbf{y}}$ definiert und kann einfach zum Graph hinzugefügt und mit differenziert werden.

Allgemeiner Fall ($W^{(i)}$ sind Matrizen)



Lösung für 2., Teil 1:

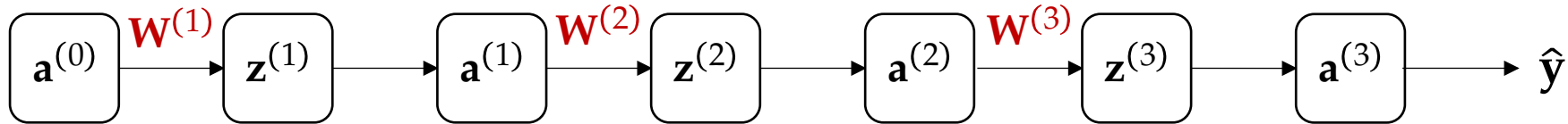
- Die Berechnung mittels **multivariater** Kettenregel geht formell genauso, (fast) alle skalaren Multiplikationen werden zu Matrix-Multiplikationen
- Die Ableitungen der einzelnen Knoten nach ihren Vorgängern (d.h. die Jacobimatrizen) sind:

$$\frac{\partial z^{(i)}}{\partial a^{(i-1)}} = W^{(i)} \text{ (Ableitung lineare Funktion)}$$

$$\frac{\partial a^{(i)}}{\partial z^{(i)}} = \text{diag} \left(g'(z^{(i)}) \right)$$

(Ableitung der elementweisen Aktivierungsfunktionen, z.B. log. Fkt., tanh, ReLU)

Allgemeiner Fall ($\mathbf{W}^{(i)}$ sind Matrizen)



Lösung für 2., Teil 2:

- Etwas aufpassen muss man bei der Ableitung $\frac{\partial \hat{y}}{\partial \mathbf{W}^{(i)}} = \frac{\partial \hat{y}}{\partial \mathbf{z}^{(i)}} \frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{W}^{(i)}}$.

Der zweite Term (Ableitung eines Vektors $\mathbf{z}^{(i)}$ nach einer Matrix $\mathbf{W}^{(i)}$) ist nicht mehr einfach

$\frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{W}^{(i)}} = \mathbf{a}^{(i-1)}$ (Vektor), sondern ein komplizierteres Objekt (ein Tensor).

- Es ergibt sich insgesamt (ohne Details):

$$\frac{\partial \hat{y}}{\partial \mathbf{W}^{(i)}} = \frac{\partial \hat{y}}{\partial \mathbf{z}^{(i)}} \frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{W}^{(i)}} = \left(\frac{\partial \hat{y}}{\partial \mathbf{z}^{(i)}} \right)^T \cdot \left(\mathbf{a}^{(i-1)} \right)^T$$

(Spaltenvektor x Zeilenvektor = Matrix, ein sog. äußeres Produkt)

Zusammenfassung Backpropagation (II)

- Neuronale Netze sind Funktionen, die als (eine bestimmte Form von) Berechnungsgraphen dargestellt werden.
- Ein Neuron kann als zwei separate Knoten dargestellt werden:
 - 1. Knoten: gewichtete Summe
 - 2. Knoten: Anwendung Aktivierungsfunktion
- Kanten repräsentieren die Gewichtung und Verkettung dieser Operationen
- Durch Anwendung der mehrdimensionalen Kettenregel lässt sich die Berechnung auch Vektor-basiert durchführen.
- Dabei benötigt man aber Tensoren, Verallgemeinerungen von Vektoren und Matrizen

Software: PyTorch

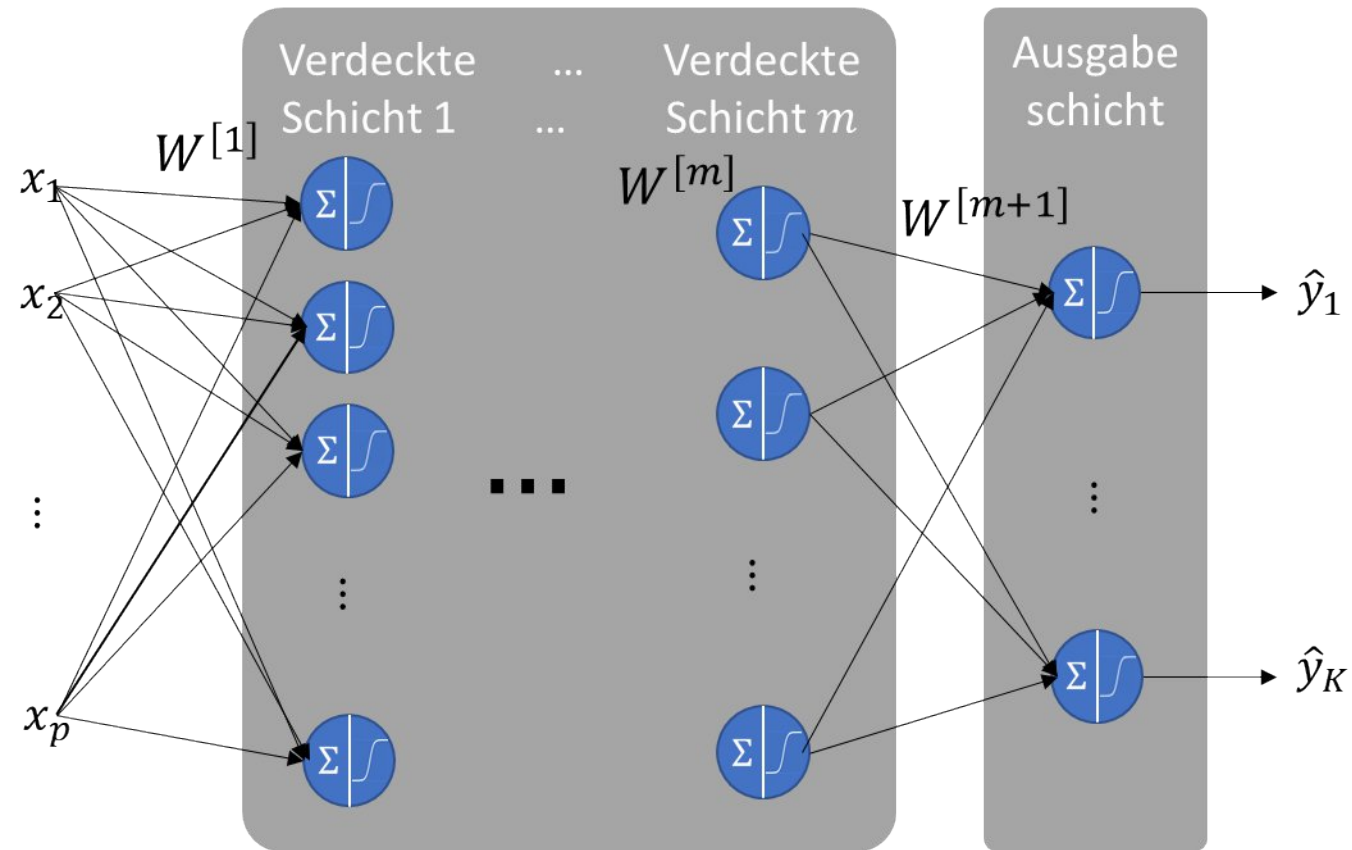
- Machine learning framework mit sehr viel Funktionalität & Ökosystem
- Das Herzstück von PyTorch bilden die torch.tensor Datenstrukturen und die autograd Funktionalität zur automatischen und effizienten Auswertung von Ableitungen

```
# An welchem Wert sollen die Funktion und die Ableitung ausgewertet werden?
x0 = 2
# Erzeugen des Tensor Datentyps. requires_grad=True bewirkt, dass alle Operationen,
# bei denen x verwendet wird, in einen Berechnungsgraphen übersetzt werden.
x = torch.tensor(x0, dtype=torch.float, requires_grad=True)
# Ein univariates Polynom
fx = x**2+1/3*x**3
fx2 = x**2+1/3*x**3+4*x
# Nun wird die Rückwärtsphase (backwards sweep) ausgeführt und die Ableitung ausgewertet.
fx.backward(retain_graph=True)
# In x.grad ist jetzt die Ableitung von df/dx gespeichert
print(x.grad)
```

Rückblick

- Neuronale Netze als **allgemeine, nichtlineare Modelle** (im Gegensatz zu linearer und logistischer Regression)
- Training durch Minimierung einer **Verlustfunktion** mittels (stochastischem) Gradientenabstieg
- Automatische Berechnung der Gradienten mittels **Backpropagation**
- Freiheitsgrade:
 - Anzahl verdeckte Schichten
 - Anzahl Neuronen pro Schicht

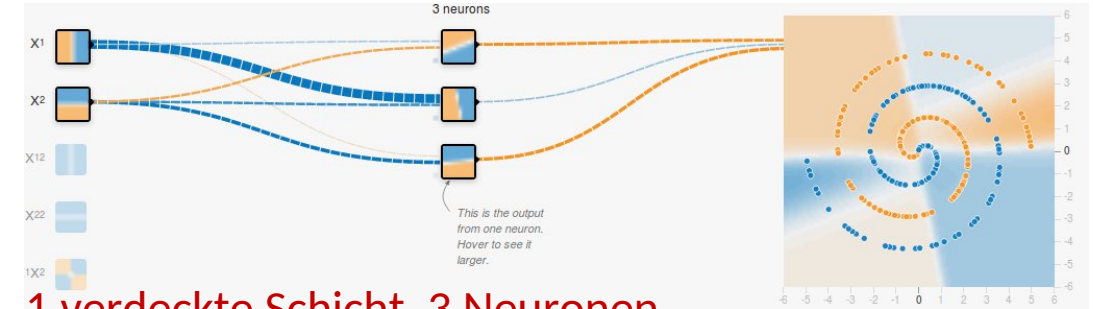
—>Wie wählen?



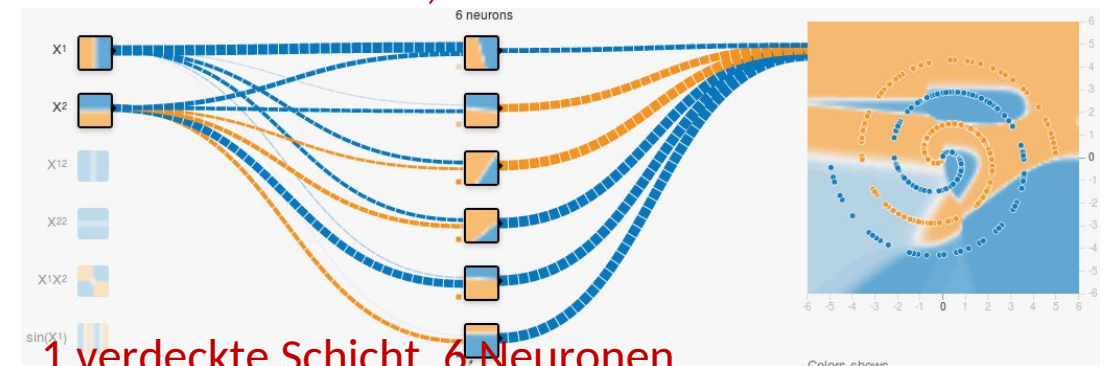
6.4 Probleme (und Lösungsansätze) beim Trainieren

Architektur neuronaler Netze

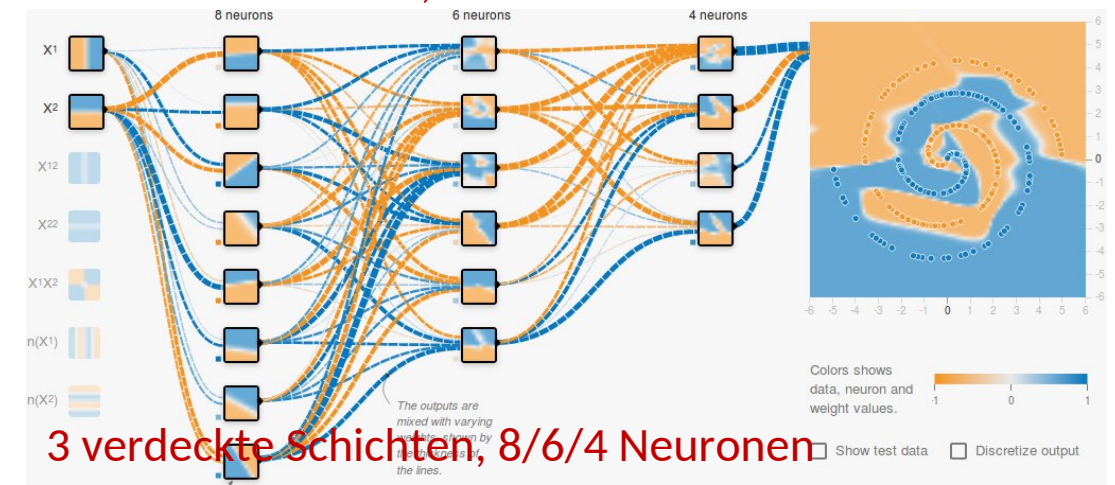
- Je mehr Schichten und Neuronen pro Schicht, desto mehr Freiheitsgrade hat das Netz – umso kompliziertere Klassifikations-/Prädiktionsfunktionen können gelernt werden
- —> Je tiefer, desto besser?
Zwei Probleme:
 1. Vanishing gradients
 2. Overfitting



1 verdeckte Schicht, 3 Neuronen



1 verdeckte Schicht, 6 Neuronen



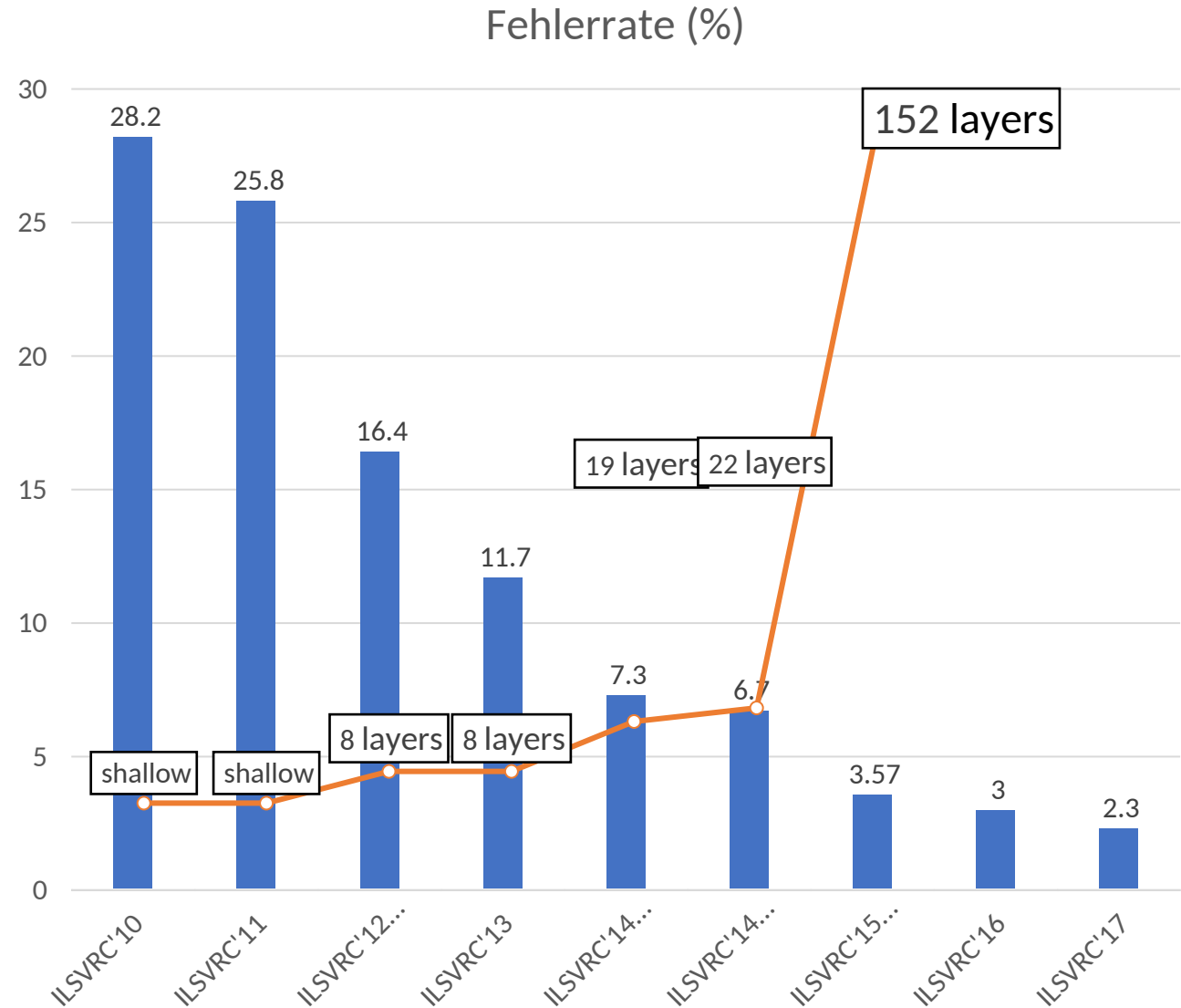
3 verdeckte Schichten, 8/6/4 Neuronen

Vanishing gradient Problem

Architektur neuronaler Netze

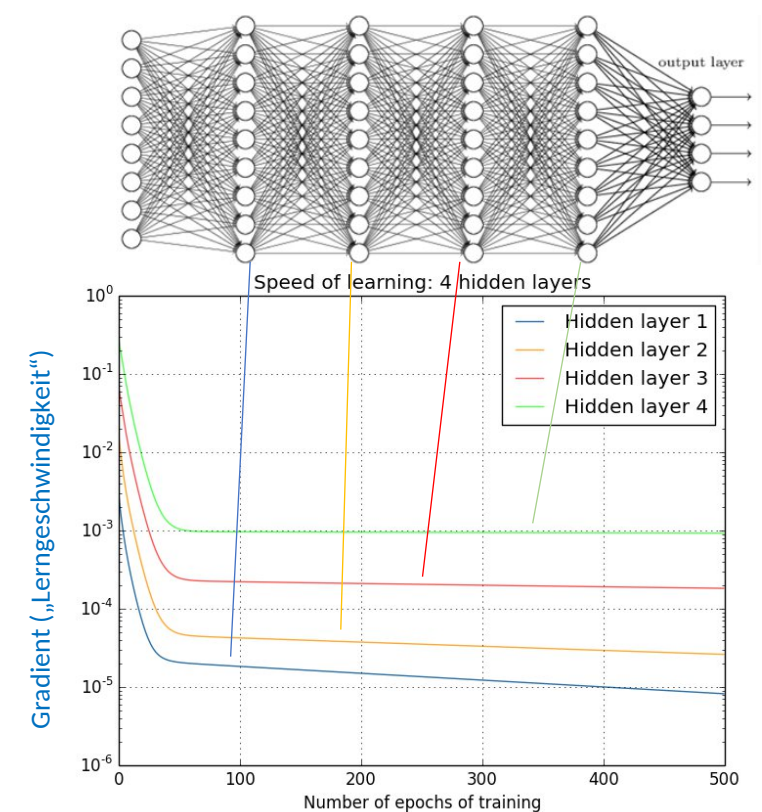
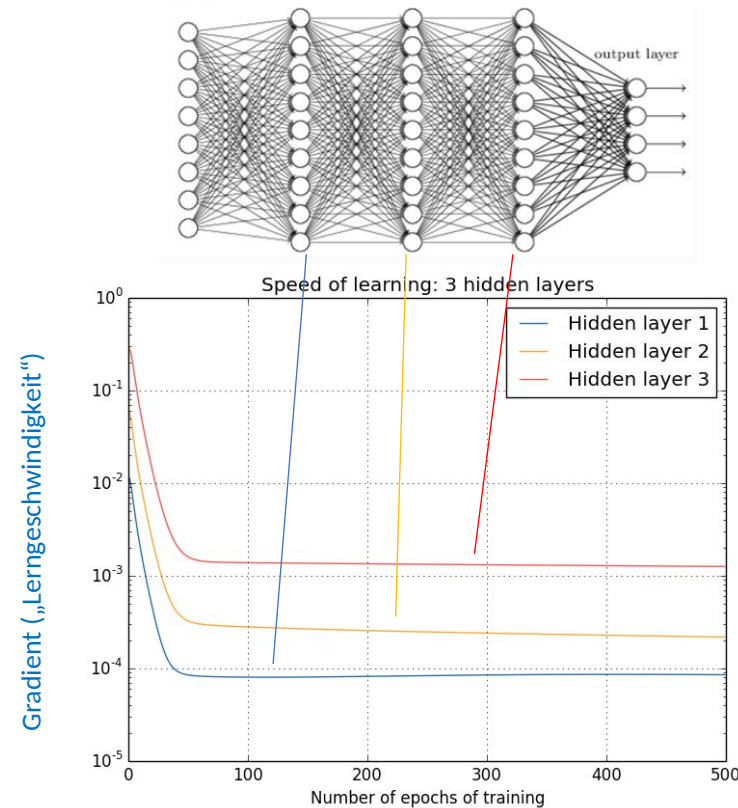
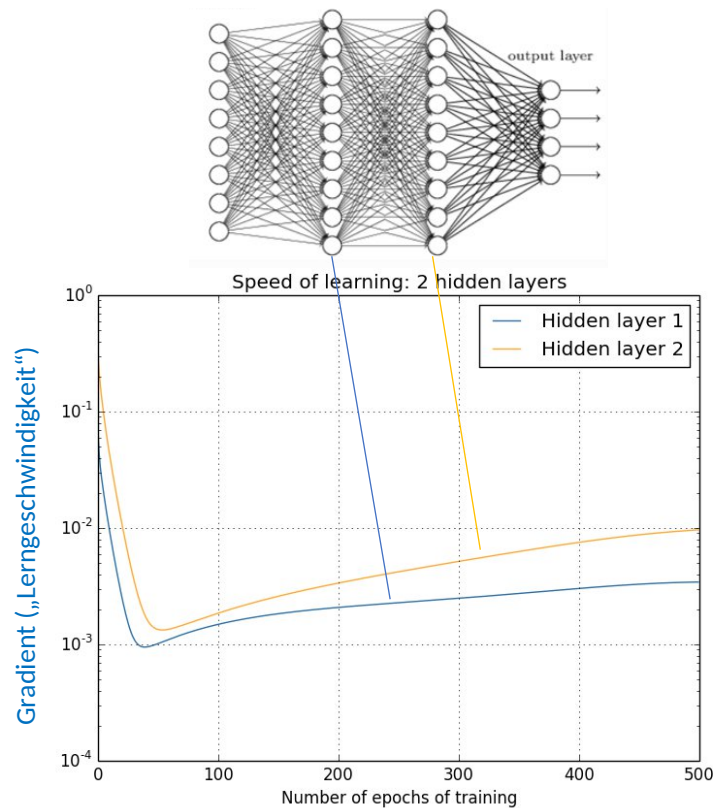
ImageNet Large Scale Visual Recognition Challenge (ILSVRC):

- Wettbewerb zur Klassifizierung von ~14 Mio Bildern in ~20000 Kategorien
- Potential von deep learning wurde besonders sichtbar
- In der Folge enorme Forschungsaktivität im Bereich deep learning (seit 2010 bis heute)



Beobachtung: Gradienten während Training

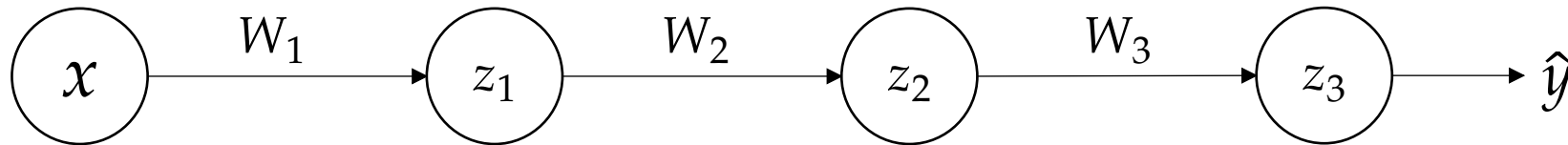
Logistische Aktivierungsfunktion, Batch Gradienten-Abstieg mit 1000 Trainingsbildern, 500 Epochen.



Gradient (= „Lerngeschwindigkeit“) ist kleiner, je mehr nachfolgende verdeckte Schichten es gibt!

Modellproblem: Analyse der Gradienten

- 1D-Beispiel: alle W_i sind Skalare



- Ableitung nach Variablen in verschiedenen Schichten:

$$\frac{\partial \hat{y}}{\partial W_3} = \sigma'(W_3 z_2) \cdot z_2$$

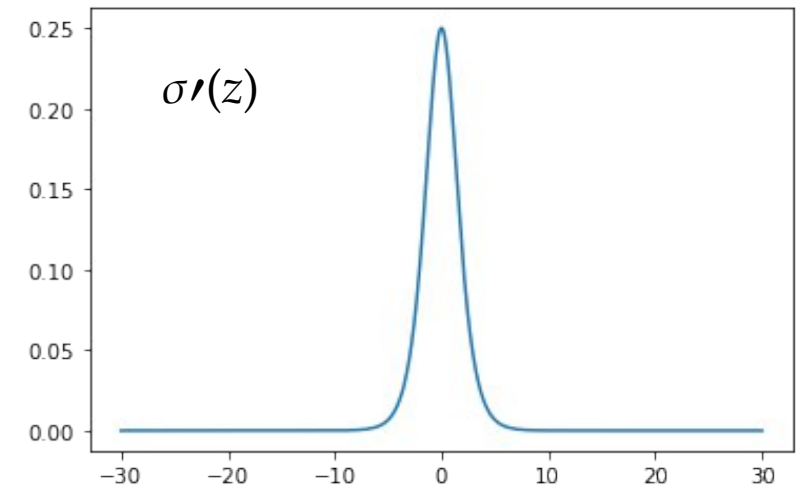
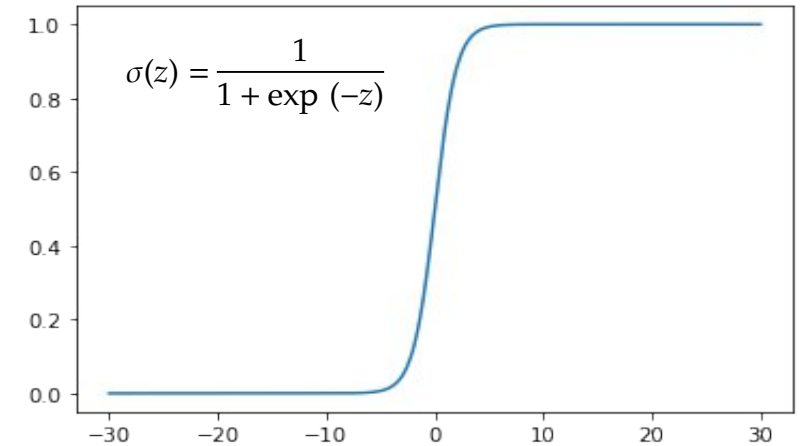
$$\frac{\partial \hat{y}}{\partial W_2} = \sigma'(W_3 z_2) \cdot W_3 \cdot \sigma'(W_2 z_1) \cdot z_1$$

$$\frac{\partial \hat{y}}{\partial W_1} = \sigma'(W_3 z_2) \cdot W_3 \cdot \sigma'(W_2 z_1) \cdot W_2 \cdot \sigma'(W_1 x) \cdot x$$

- Beobachtung: Ableitung der Aktivierungsfunktion wird potenziert, je weiter vorne die Variable im Netz ist.

Verswindende Gradienten

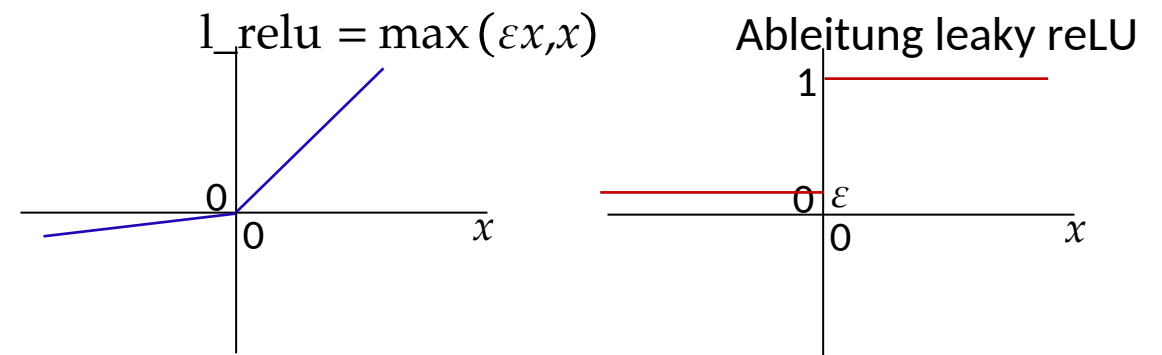
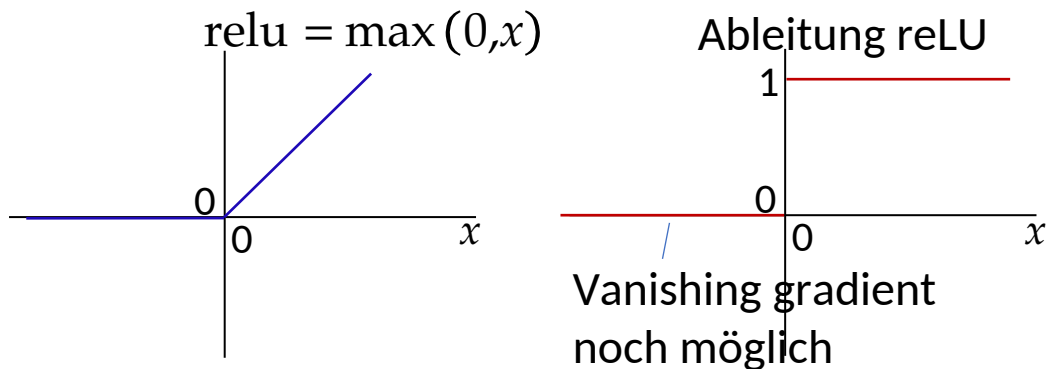
- Problem: σ' ist fast überall sehr nahe bei Null!
—> $\sigma'(\dots) \cdot \sigma'(\dots) \cdot \dots \cdot \sigma'(\dots)$ wird in der Regel kleiner, wenn die Anzahl der Schichten erhöht wird
- Konsequenz: Der Gradient wird sehr klein, das Gradientenverfahren verändert die Variable also kaum.
- Dies ist ein fundamentales Problem beim Training von **tiefen** neuronalen Netzen! (sog. „vanishing gradient“ Problem)



Lösungsansatz: Andere Aktivierungsfunktionen

$$\text{relu: } \text{relu}(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases} = \max(0, x)$$

$$\text{leaky relu: } \text{l_relu} = \max(\varepsilon x, x)$$



Achtung: hier gibt es u.U. das umgekehrte Problem: Gradienten wachsen unkontrolliert („exploding gradient“), kann mittels Regularisierung entgegengewirkt werden.

Lösungsansatz: Adaptive Lernraten

Varianten von stochastischem Gradientenabstieg mit adaptiver Lernrate (Adagrad, RMSProp, Adam) berechnen in jeder Iteration automatisch eine **eigene Lernrate für jeden Parameter w** .

Beispiel Adagrad:

Die Iterationsvorschrift für die i -te Komponente des Vektor w lautet:

Lernrate gesamt

$$\mathbf{w}_i^{[k+1]} = \mathbf{w}_i^{[k]} - \frac{\alpha^{[k]} \nabla L(\mathbf{w}^{[k]})}{\epsilon + \sqrt{s_i^{[k]}}}$$

$\alpha^{[k]} \in \mathbb{R}$ ist eine Art „Basislernrate“ (für alle Komponenten gleich).

Wg. Division durch $\sqrt{s_i}$ sind Lernraten für einzelne Parameter umgekehrt proportional zum Betrag ihrer Ableitung.

Wobei $s^{[k]}$ ein Vektor ist, dessen i -ter Eintrag die Summe der Quadrate der partiellen Ableitungen $\frac{\partial L(\mathbf{w})}{\partial w_i}$ ist (ausgewertet an den Punkten $w^{[1]}, w^{[2]}, \dots, w^{[k]}$):

$$s_i^{[k]} = \sum_{j=1}^k \left(\frac{\partial L(\mathbf{w}^{[j]})}{\partial w_i} \right)^2$$

Die partielle Ableitung nach der i -ten Variable im j -ten Schritt des Verfahrens

ϵ ist ein Parameter (z.B. 10^{-8}), der verhindert, dass durch Null geteilt wird.

Regularisierung

Techniken zur Regularisierung: Strafterm (= „Weight decay“)

- Regularisierung: Überanpassung (Overfitting) des Modells an die Trainingsdaten verhindern, indem beim Trainieren die Modellflexibilität eingeschränkt wird.
- Möglichkeit 1: Addieren von Strafterm zur Verlustfunktion (wie bei linearer/logist. Regression!). Statt

$$\min_{\mathbf{w}} L(\mathbf{w}) = \sum_{i=1}^N l(y_i, f(\mathbf{x}_i; \mathbf{w}))$$

optimieren wir:

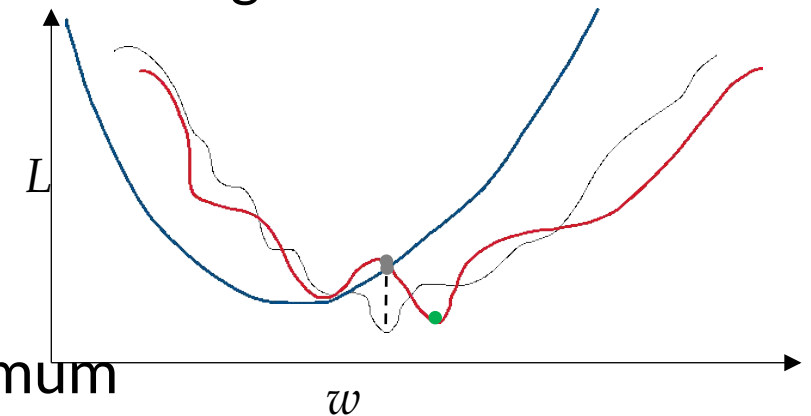
$$\min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_{i=1}^N \mathbf{w}_i^2 = \sum_{i=1}^N l(y_i, f(\mathbf{x}_i; \mathbf{w})) + \lambda \sum_{i=1}^N \mathbf{w}_i^2$$

bzw.

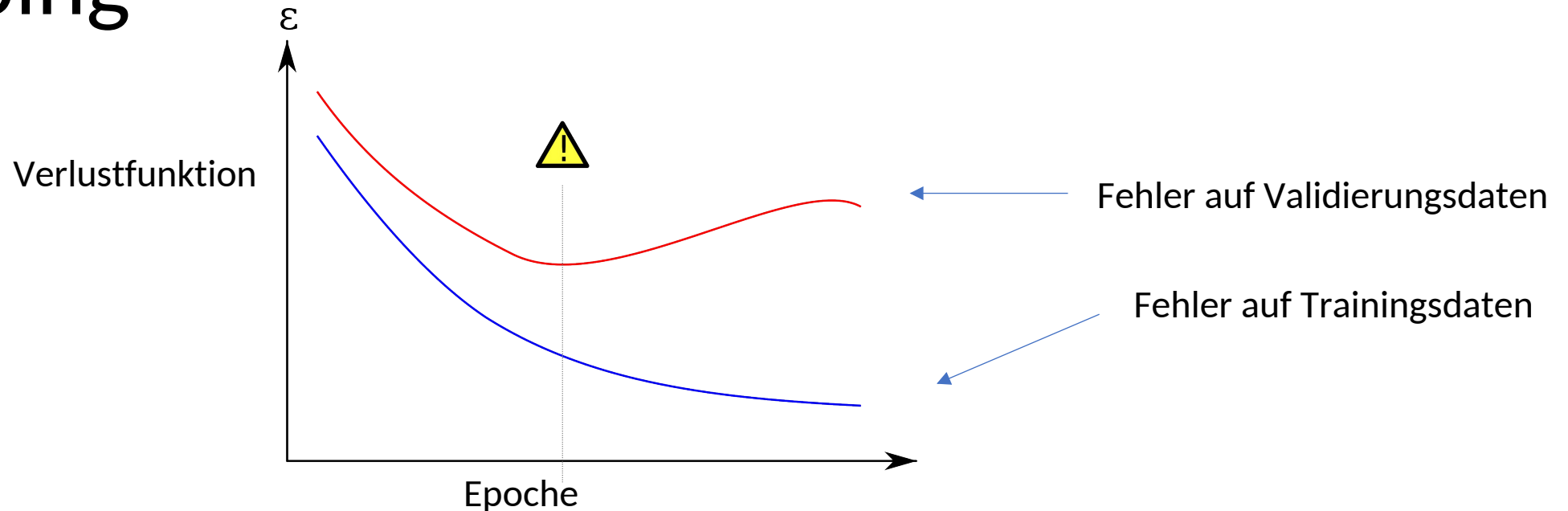
$$\min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_{i=1}^N |\mathbf{w}_i| = \sum_{i=1}^N l(y_i, f(\mathbf{x}_i; \mathbf{w})) + \lambda \sum_{i=1}^N |\mathbf{w}_i|$$

Techniken zur Regularisierung: **S**GD

- Neben der beschleunigten Rechenzeit hat SGD einen weiteren entscheidenden Vorteil: SGD wirkt regularisierend, d.h. verhindert Overfitting!
- Erinnerung: In jeder Iteration wird eine andere Summe ausgewertet (bzw. die Ableitung), das entspricht jedes Mal einer leicht veränderten Funktion.
- Dies verhindert, dass das Verfahren zu einem Minimum konvergiert, das hauptsächlich die Eigenheiten des Trainingsdatensatzes repräsentiert.



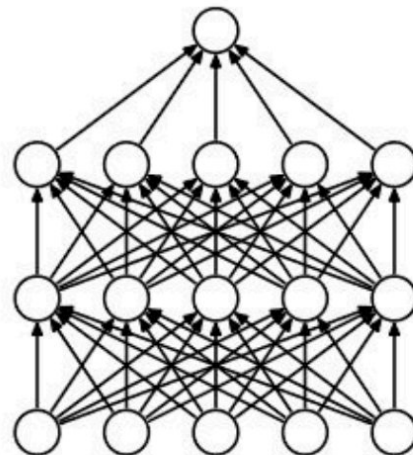
Techniken zur Regularisierung: Early Stopping



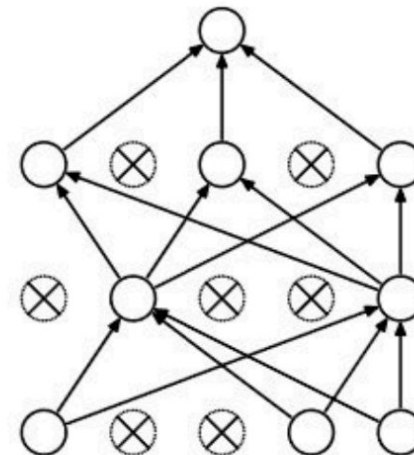
- Idee: Stoppe das Verfahren, wenn es noch nicht im Minimum ist (Fehler auf Trainingsdaten verringert sich also noch), aber der Fehler auf Validierungsdaten zu steigen beginnt —> Overfitting
- Umsetzung (z.B.): Überwache Fehler auf Validierungsdaten, stoppe, wenn sich der Fehler z.B. M Epochen lang nicht verringert (alte Iterierte müssen zwischengespeichert werden)

Techniken zur Regularisierung: Dropout

- Idee: In jeder Iteration des SGD, setze eine **zufällige Teilmenge von Neuronen (Aktivierungen) gleich Null**
- Motivation: Das Netz lernt so robustere interne Darstellungen (Output der verdeckten Schichten) ohne dabei einzelnen Neuronen zu viel Gewicht beizumessen



(a) Standard Neural Net



(b) After applying dropout.

Techniken zur Regularisierung: Dropout

- Während des Trainings: In jeder Batch wird der Output jedes Neurons mit Wahrscheinlichkeit p auf Null gesetzt (in der Praxis p zwischen 0.2 und 0.5)
- Netz wird weniger sensitiv gegenüber einzelnen Parametern.
- Dropout wird nur während des Trainings angewendet. Bei der Evaluierung wird das volle Netz ohne Dropout genutzt.
- Achtung: Während des Trainings werden die Aktivierungen mit $\frac{1}{1-p}$ skaliert – so ist die Summe der Outputs einer Schicht während Training und Evaluierung (im Mittel) gleich

Zusammenfassung: Hyperparameter für Neuronale Netze

Allein beim Training von neuronalen Netzen gibt es u.a. folgende Hyperparameter:

- Lernrate (inkl. Strategie zur Anpassung der Lernrate)
- Parameter für weight decay
- Batchgröße
- Wahrscheinlichkeit für Dropout

Außerdem: Freiheitsgrade bei der Entwicklung der Architektur, u.a.:

- Anzahl Schichten
 - Anzahl Neuronen
 - Aktivierungsfunktion
 - Architektur, d.h. Verbindungen/Arten der Schichten (—> kommt noch)
- > Enorme Anzahl an „Stellschrauben“ für ein Modell!**